

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE MATEMATICHE, FISICHE E
NATURALI



CORSO DI LAUREA IN
INFORMATICA

TESI DI LAUREA IN
ALGORITMI E STRUTTURE DATI

TITOLO

Progettazione e Sviluppo di un Sistema di Autenticazione Passwordless
Basato su Protocolli a Conoscenza Zero e Firme di Schnorr

Relatore:

Prof.

Francesco Pasquale

Candidato:

Ionut Georgian Zbirciog

Matricola: *0308984*

Anno Accademico 2024/2025

INTRODUZIONE	5
1 Elementi di Aritmetica Modulare e Fondamenti di Crittografia	8
1.1 Aritmetica Modulare	8
1.1.1 Gruppi	9
1.1.2 Gruppi Ciclici ed Elementi Primitivi	10
1.2 Logaritmo Discreto	10
1.3 AES-GCM	12
1.4 SHA-256	12
2 Schnorr: Identificazione e Firma Digitale	14
2.1 Protocolli "Zero Knowledge Proof"	14
2.1.1 Proprietà	16
2.2 Protocolli Sigma	17
2.2.1 Proprietà	18
2.3 Firme Digitali	21
2.3.1 Euristica di Fiat-Shamir per le Firme	22
2.4 Protocollo d'Identificazione di Schnorr	23
2.4.1 Proprietà	24
2.4.2 Schema di Firma Digitale di Schnorr	26
3 Progetto Sperimentale - SchnorrAuthApp	27
3.1 Analisi Architetturale	28
3.1.1 Scelta e Definizione del Database	28
3.1.2 Calcolo dell'Hash delle Chiavi Pubbliche	31
3.1.3 Salvataggio della Chiave Segreta in Locale	31
3.1.4 SchnorrProver e SchnorrVerifier	31
3.2 Funzionamento del Protocollo SAP	32
3.2.1 Fase di Handshake	32
3.2.2 Fase di Registrazione/Autenticazione	34
3.2.3 Processo di Accoppiamento di un Dispositivo	37

3.3	Applicazione Client Flutter	40
3.3.1	F-Droid	40
3.3.2	Livelli di Sicurezza	41
3.3.3	Interfaccia Grafica	42
4	Conclusioni e Sviluppi Futuri	45
4.1	Possibili Sviluppi Futuri	46
	BIBLIOGRAFIA	48
A	APPENDICE	49
A.1	KeyManager	49
A.2	SchnorrProver e SchnorrVerifier	50

INTRODUZIONE

I sistemi di autenticazione più utilizzati attualmente sono quelli password-based. L'accesso a un account – che si tratti di *Instagram*, *Gmail* o di un portale universitario – richiede l'inserimento di un identificativo (username, email o matricola) abbinato a una **password**, ovvero una stringa alfanumerica creata al momento della registrazione.

In termini di *user experience*, l'autenticazione basata su password risulta la soluzione più agevole, poiché richiede all'utente di memorizzare esclusivamente la password scelta in fase di registrazione. Nonostante ciò, il sistema è esposto a vulnerabilità: dalla perdita o dimenticanza della password, che impone una procedura di reimpostazione, fino alla possibilità che un attaccante la indovini (attacchi brute-force/enumerazione) o la intercetti, ottenendo accesso non autorizzato all'account.

La presente tesi ha come obiettivo la progettazione di un sistema (client-server) di autenticazione privo di password, fondato sull'utilizzo di un protocollo **Zero Knowledge Proof**, concetto introdotto per la prima volta alla fine degli anni '80 da Shafi Goldwasser, Silvio Micali e Charles Rackoff nel documento *The Knowledge Complexity of Interactive Proof-Systems* [1]. In particolare, verrà trattato il **Protocollo di Identificazione di Schnorr** e la relativa **Firma Digitale**, ideato da Claus Peter Schnorr alla fine degli anni '80 nel documento *Efficient signature generation by smart cards* [2]. A differenza dei meccanismi password-based, l'approccio *ZKP* elimina la necessità di comunicare la password a ogni accesso, riducendo sia la probabilità di dimenticanza da parte dell'utente sia l'esposizione a rischi di sicurezza.

All'interno del protocollo d'identificazione di Schnorr, ciascun client dispone di una coppia di chiavi: la chiave pubblica ($pk := u$) e la chiave segreta ($sk := \alpha$). Per garantire la sicurezza del protocollo, sia contro attacchi diretti sia contro tentativi di intercettazione, vengono impiegate operazioni modulari all'interno del gruppo crittografico $\mathbb{Z}_p$, con p scelto come numero primo. I gruppi utilizzati seguono le specifiche dello standard RFC 3526 [3], il che implica che le chiavi avranno una lunghezza variabile a seconda della classe scelta (fino a 2048 bit nel caso del gruppo modp-2048 , e così via).

Un punto cruciale è l'assunzione che il problema del logaritmo discreto (DL) sia computazionalmente **difficile** da risolvere. Si tratta di un problema matematico che, pur essendo facile da eseguire in una direzione, risulta estremamente complicato da inver-

tire. Dato un gruppo moltiplicativo ciclico G di ordine primo q , un generatore $g \in G$ e un elemento $u \in G$, il problema del logaritmo discreto consiste nel determinare l'esponente $\alpha \in \{0, \dots, q-1\}$ tale che

$$u = g^\alpha, \text{ dove } \alpha = \log_g u,$$

dove α è appunto chiamato **logaritmo discreto** di u .

Poiché questo problema è considerato intrattabile con gli attuali metodi noti anche qualora un malintenzionato riuscisse a intercettare la chiave pubblica ($pk := g^\alpha$), non sarebbe comunque in grado di risalire in tempi utili alla corrispondente chiave privata ($sk := \alpha$).

Nel contesto di un protocollo di autenticazione si distinguono due entità principali: il **Prover** e il **Verifier**. Il *Prover* rappresenta il client, ossia l'entità software che consente all'utente di autenticarsi, mentre il *Verifier* rappresenta il server, il quale ha il compito di validare tale autenticazione. In uno scenario di autenticazione *passwordless* non viene utilizzata alcuna password tradizionale. Al contrario, il *Prover* deve dimostrare al *Verifier* di possedere una determinata chiave segreta associata al proprio account, ma senza mai trasmetterla direttamente.

Il sistema sviluppato in questa tesi è basato sul **Protocollo di Autenticazione e Firma di Schnorr**. L'obiettivo è fornire un meccanismo di accesso sicuro che permetta all'utente di autenticarsi utilizzando esclusivamente il proprio *username*, dimostrando il possesso della chiave segreta senza trasmetterla. Il sistema è implementato secondo un'architettura client-server, con un server sviluppato in *Python* e un database *MongoDB*, e prevede la gestione delle chiavi tramite operazioni di hashing (SHA-256) e cifratura (AES-GCM). Il protocollo di comunicazione, denominato **SAP (Schnorr-based Authentication Protocol)**, si articola in tre fasi principali: *handshake*, *registrazione/autenticazione* e *accoppiamento di dispositivi*.

Nonostante l'elevato livello di sicurezza garantito da questo approccio, nei sistemi reali si tende a preferire l'autenticazione tradizionale tramite password, ritenuta più agevole dal punto di vista della *user experience*, pur a fronte della necessità di memorizzare la password stessa. Una delle principali criticità dei sistemi *passwordless* risiede nel fatto che la chiave segreta è *device-dependent/client-dependent*: essa deve infatti risiedere sul dispositivo o sul client utilizzato per l'accesso. Di conseguenza, la cancellazione accidentale della chiave da parte dell'utente comporterebbe l'impossibilità di autenticarsi. Uno degli obiettivi del sistema presentato in questa tesi è quello di mitigare tale limite introducendo un meccanismo di associazione di più dispositivi, sfruttando la **Firma Digitale di Schnorr**.

In conclusione, è stata realizzata un'applicazione multi-platform (Android, Linux, Windows) con *Flutter*, che, tramite un'interfaccia grafica, rende più semplice l'utilizzo del sistema di autenticazione, mentre il client Linux sviluppato in *Python* è disponibile esclusivamente in versione CLI (riga di comando).

La struttura del lavoro svolto è organizzata come segue:

- **Capitolo 1 - Elementi di Aritmetica Modulare e Fondamenti di Crittografia**

In questo capitolo vengono presentate le nozioni fondamentali di aritmetica modulare (definizione di gruppo, sottogruppo, etc...), insieme ad alcuni concetti chiave della crittografia, come il problema del logaritmo discreto, che costituiscono la base teorica del lavoro.

- **Capitolo 2 - Schnorr: Identificazione e Firma Digitale**

Viene presentato il concetto di protocollo *Zero-Knowledge Proof* e, successivamente, viene analizzato nel dettaglio il protocollo di identificazione di Schnorr e il relativo schema di firma digitale.

- **Capitolo 3 - Progetto Sperimentale - SchnorrAuthApp**

È dedicato allo sviluppo del sistema di autenticazione *passwordless* progettato nell'ambito di questa tesi, basato sul protocollo di identificazione e firma di Schnorr, con descrizione dell'architettura, del protocollo di comunicazione e dell'applicazione Flutter sviluppata.

- **Capitolo 4 - Conclusioni e Sviluppi Futuri**

Riassume i risultati ottenuti e presenta alcune considerazioni sulle possibili estensioni e sviluppi futuri del progetto.

- **Appendice**

Contiene frammenti di codice e implementazioni ritenute significative per la comprensione del sistema sviluppato.

Capitolo 1

Elementi di Aritmentica Modulare e Fondamenti di Crittografia

In questo capitolo vengono presentate le nozioni fondamentali di aritmetica modulare insieme ad alcuni concetti chiave della crittografia presenti nel libro *Cryptography Theory and Practice* [4], che costituiranno la base teorica per l'analisi del protocollo di identificazione di Schnorr e per la progettazione del sistema di autenticazione che su di esso si fonda.

1.1 Aritmetica Modulare

Definizione 1.1.1 (Congruenza). Siano a e $b \in \mathbb{Z}$ e sia $m \in \mathbb{Z}^+$.

Scriviamo $a \equiv b \pmod{m} \Leftrightarrow m$ divide $b - a$ e si legge " a è **congruente** a b modulo m ".

Definizione 1.1.2 (Riduzione Modulare). Supponiamo di dividere a e b per m , ottenendo quoziente e resto intero. Abbiamo quindi:

$$a = q_1 m + r_1 \text{ e } b = q_2 m + r_2, \text{ dove } 0 \leq r_i \leq m - 1 \text{ e } i \in \{1, 2\}.$$

Allora, $a \equiv b \pmod{m} \Leftrightarrow r_1 = r_2$. Scriveremo r_1 come $a \pmod{m}$, ovvero il resto della divisione tra a e m e si legge " a è **ridotto modulo** m ".

Definizione 1.1.3 (Aritmetica Modulo m). Definiamo aritmetica modulo m come:

$$\mathbb{Z}_m = \{0, \dots, m - 1\}$$

insieme a due operazioni, $+$ (addizione) e $\cdot$ (moltiplicazione). Tali operazioni funzionano esattamente come la vera addizione e moltiplicazione ad eccezione dei risultati che sono ridotti modulo m . Ad esempio, se $m = 5$, allora $3 + 4 \equiv 2 \pmod{5}$ e $3 \cdot 4 \equiv 2 \pmod{5}$.

1.1.1 Gruppi

Definizione 1.1.4 (Gruppo). Un **gruppo** è un coppia $G = (X, \star)$ dove X è un insieme e $\star$ è l'operazione binaria definita su X , che soddisfa le seguenti proprietà:

- L'operazione $\star$ è **associativa**: $\forall a, b, c \in X, (a \star b) \star c = a \star (b \star c)$;
- Esiste l'elemento $id \in X$ chiamato **identità**, tale che $a \star id = id \star a = a, \forall a \in X$;
- $\forall a \in X, \exists b \in X$ chiamato **inverso** di a , tale che $a \star b = b \star a = id$.

Definizione 1.1.5. Un gruppo $G = (X, \star)$ è **abeliano** se l'operazione $\star$ è **commutativa**, ovvero $a \star b = b \star a, \forall a, b \in X$.

Definizione 1.1.6. Un gruppo $G = (X, \star)$ è **finito** se l'insieme X è finito.

Definizione 1.1.7. L'**ordine** di un gruppo finito $G = (X, \star)$, indicato con $ord(G)$, è uguale a $|X|$.

Definizione 1.1.8 (Sottogruppo). Sia $G = (X, \star)$ un gruppo finito e sia $Y \subseteq X$. Allora diremo che $H = (Y, \star)$ è un **sottogruppo** di G se H è anch'esso un gruppo finito.

Teorema 1.1.1. Sia $G = (X, \star)$ un gruppo finito e sia $Y \subseteq X$. Allora diremo che $H = (Y, \star)$ è un **sottogruppo** di G se H è **chiuso** rispetto l'operazione $\star$, ovvero se $\forall h_1, h_2 \in H, h_1 \star h_2 \in H$.

Teorema 1.1.2 (Teorema di Lagrange). Sia $H = (Y, \star)$ un sottogruppo finito del gruppo $G = (X, \star)$. Allora $ord(H)$ divide $ord(G)$.

Definizione 1.1.9 (Gruppo additivo $\mathbb{Z}_n$). Sia $n \geq 2$ un intero. Allora la coppia $(\mathbb{Z}_n, +)$ è un gruppo abeliano finito di ordine n , dove il $+$ indica l'operazione di addizione modulo n . L'elemento identità è 0 e l'inverso di un elemento $a \in \mathbb{Z}_n$ è $(-a) \pmod{n}$.

Definizione 1.1.10 (Gruppo moltiplicativo $\mathbb{Z}_p^*$). Sia $p \geq 2$ primo. Definiamo $\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$. Allora $(\mathbb{Z}_p^*, \cdot)$ è un gruppo finito abeliano di ordine $p - 1$, dove $\cdot$ indica l'operazione di moltiplicazione modulo p . L'elemento identità è 1 e l'inverso di un elemento $a \in \mathbb{Z}_p^*$ è a^{-1} .

Definizione 1.1.11 (Ordini degli Elementi del Gruppo). Dato un gruppo finito $(X, \star)$, definiamo **ordine** di un elemento $a \in X$, indicato con $ord(a)$, il più piccolo intero m tale che

$$\underbrace{a \star a \star \dots \star a}_m = id$$

In base alla tipologia di operazione che assume $\star$, si ha

$$\underbrace{a \star a \star \dots \star a}_m = \begin{cases} a^m & \text{se l'operazione è scritta in forma moltiplicativa,} \\ am & \text{se l'operazione è scritta in forma additiva.} \end{cases}$$

L'elemento identità ha ordine 1 mentre qualsiasi altro elemento diverso dall'identità ha ordine maggiore di 1. Per esempio, nel gruppo $(\mathbb{Z}_n, +)$, l'elemento 1 ha ordine n .

1.1.2 Gruppi Ciclici ed Elementi Primitivi

Definizione 1.1.12 (Gruppo Ciclico). Un gruppo abeliano finito $(X, \star)$ è un **gruppo ciclico** se esiste un elemento $a \in X$ avente ordine uguale a $|X|$. Tale elemento è chiamato **generatore** del gruppo.

Ad esempio, sia $p \geq 2$ primo. Allora $(\mathbb{Z}_p^*, \cdot)$ è un gruppo ciclico di ordine $p - 1$, e un generatore di questo gruppo è chiamato **elemento primitivo modulo p** .

In altre parole, esiste $g \in \mathbb{Z}_p^*$, tale che $\mathbb{Z}_p^* := \{1, g, g^2, g^3, \dots, g^{p-2}\}$.

Per esempio, in $\mathbb{Z}_7^* : \langle 3 \rangle := \{1, 3, 3^2, 3^3, 3^4, 3^5\} = \{1, 3, 2, 6, 4, 5\} \pmod{7} = \mathbb{Z}_7^*$.

Definizione 1.1.13. Sia $G = (X, \star)$ un gruppo finito e sia $a \in X$. Definiamo

$$\langle a \rangle := \{a^i : i \geq 0\}.$$

È facile da dimostrare che $(\langle a \rangle, \star)$ è un sottogruppo di G , che è ciclico e che $\text{ord}(\langle a \rangle) = \text{ord}(a)$. Il sottogruppo $(\langle a \rangle, \star)$ è chiamato **sottogruppo generato da a** e per il Teorema 1.1.2, $\text{ord}(\langle a \rangle) \mid \text{ord}(G)$.

1.2 Logaritmo Discreto

In questa sezione si va a definire il problema del logaritmo discreto (DL Problem) e del perché è fondamentale per la maggior parte dei protocolli di crittografia. Iniziamo con definire tale problema per un gruppo moltiplicativo finito generico $(G, \cdot)$. Scelto un elemento (generatore) $\alpha \in G$ avente ordine n , definiamo:

$$\langle \alpha \rangle := \{\alpha^i : 0 \leq i \leq n - 1\}$$

È facile da dimostrare che $\langle \alpha \rangle$ è un sottogruppo di G , che è ciclico ed ha ordine n .

Più nello specifico, il gruppo moltiplicativo maggiormente usato è $(\mathbb{Z}_p^*, \cdot)$ con p primo e ordine q anch'esso primo, tale che $p - 1 \equiv 0 \pmod{q}$. Si sceglie poi un generatore $\alpha \in \mathbb{Z}_p^*$ avente ordine primo q nel seguente modo: preso un qualsiasi elemento primitivo in $\mathbb{Z}_p^*$ lo si eleva alla $\frac{(p-1)}{q}$ potenza, determinando:

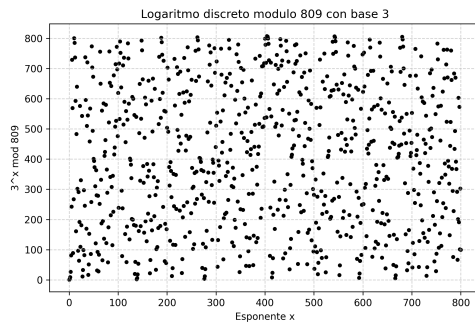
$$\langle \alpha \rangle := \{\alpha^i : 0 \leq i \leq q - 1\}$$

Definizione 1.2.1 (Logaritmo Discreto). Sia $(G, \cdot)$ un gruppo moltiplicativo finito, sia $\alpha \in G$ avente ordine n e sia $\beta \in \langle \alpha \rangle$. Il problema del **logaritmo discreto** consiste nel trovare l'unico intero a , $0 \leq a \leq n - 1$ tale che

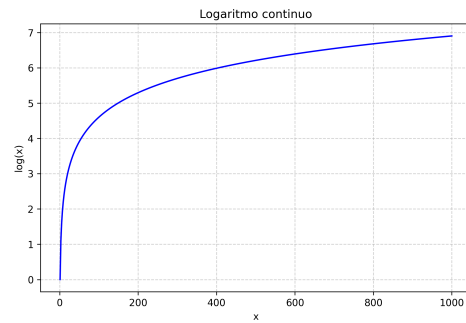
$$\alpha^a = \beta$$

ovvero $a = \log_\alpha \beta$ ed è appunto chiamato **logaritmo discreto** di β .

L'ampio utilizzo del logaritmo discreto in crittografia deriva dal fatto che il problema di calcolarlo è considerato computazionalmente difficile, cioè non si conoscono algoritmi efficienti in grado di risolverlo in tempi ragionevoli. Al contrario, l'operazione inversa, ossia l'elevamento a potenza modulo un numero, può essere eseguita in maniera estremamente rapida grazie ad algoritmi ottimizzati come l'esponenziazione modulare. Questa asimmetria tra la facilità del calcolo diretto e la difficoltà apparente del problema inverso è ciò che conferisce sicurezza a molti protocolli crittografici basati sul logaritmo discreto.



(a) Grafico del logaritmo discreto.



(b) Grafico del logaritmo continuo.

Figura 1.1: Confronto tra logaritmo discreto e logaritmo continuo.

Il motivo principale per cui il logaritmo discreto risulta difficilmente calcolabile, a differenza del logaritmo continuo, risiede nella struttura discreta e ciclica del gruppo modulare. Come mostrato in Figura 1.1a, i punti ottenuti mediante l'esponenziazione modulare risultano distribuiti in maniera apparentemente casuale all'interno del piano cartesiano, senza seguire un andamento regolare o monotono. La funzione esponenziale modulo p , infatti, non è monotona e presenta un comportamento periodico, il che rende impossibile definire un'inversa diretta e, di conseguenza, calcolare agevolmente il logaritmo discreto.

1.3 AES-GCM

L'**AES (Advanced Encryption Standard)**, conosciuto anche come **Rijndael**, sviluppato da J. Daemen and V. Rijmen, il cui funzionamento è descritto nel libro *The Design of Rijndael: AES - The Advanced Encryption Standard* [5], è un algoritmo di cifratura simmetrica a blocchi, standardizzato dal NIST (National Institute of Standards and Technology) nel 2001 per sostituire il precedente DES (Data Encryption Standard). È oggi uno degli algoritmi crittografici più utilizzati al mondo, grazie alla sua sicurezza e all'efficienza sia su hardware che su software.

Rijndael è una rete a sostituzione e permutazione. È veloce sia se sviluppato in software sia se sviluppato in hardware, è relativamente semplice da implementare, richiede poca memoria e offre un buon livello di protezione, motivi che complessivamente l'hanno reso il preferito rispetto agli altri algoritmi proposti. Nell'AES il blocco è di dimensione fissa (128 bit) e la chiave può essere di 128, 192 o 256 bit.

L'**AES-GCM (Galois/Counter Mode)**, descritto nello standard RFC 5288 *AES Galois Counter Mode (GCM) Cipher Suites for TLS* [6], è una modalità di utilizzo di AES che fornisce cifratura autenticata. Questo significa che garantisce sia confidenzialità, ossia i dati sono cifrati con AES in modalità CTR (Counter Mode), oltre a garantire integrità e autenticità, ovvero un tag di autenticazione assicura che i dati non siano stati alterati. AES-GCM è oggi una delle modalità più diffuse, usata in protocolli come SSH, QUIC/HTTP3, e nel Wi-Fi moderno.

1.4 SHA-256

Una **funzione di hash crittografica (CHF)** è un algoritmo che associa un input arbitrario a un output di lunghezza fissa n bit, garantendo proprietà fondamentali per la sicurezza:

- **Uniformità:** ogni valore di hash ha probabilità 2^{-n} di comparire.
- **Resistenza alla preimmagine:** dato un hash, è computazionalmente infattibile ricavare l'input corrispondente ($\approx 2^n$ tentativi).
- **Resistenza alla seconda preimmagine:** data una stringa, è difficile trovarne un'altra con lo stesso hash.
- **Resistenza alle collisioni:** è improbabile trovare due input distinti con lo stesso hash; la complessità attesa è $2^{n/2}$.

SHA-256 (Secure Hash Algorithm 256-bit) [7] è una funzione di hash crittografica appartenente alla famiglia **SHA-2**, progettata dalla National Security Agency (NSA) e pubblicata dal NIST nel 2001 come successore di **SHA-1**.

SHA-256 produce un digest di 256 bit, ossia una stringa di 64 caratteri esadecimale, indipendentemente dalla dimensione del messaggio in ingresso. Questa proprietà consente di trattare l'hash come una sorta di "impronta digitale" univoca del dato, utile per verificarne l'integrità e l'autenticità senza dover conservare l'intero contenuto originale.

Dal punto di vista operativo, l'algoritmo segue una struttura tipica delle funzioni di hash della famiglia Merkle–Damgård: al messaggio viene aggiunto inizialmente del padding e suddiviso in blocchi di 512 bit, che vengono elaborati iterativamente da una funzione di compressione basata su operazioni bitwise, somme modulari e funzioni non lineari. Alla fine di questa elaborazione, lo stato interno dell'algoritmo viene trasformato nell'hash finale di 256 bit.

Grazie a questa costruzione, SHA-256 è oggi uno degli algoritmi più utilizzati in ambito crittografico. Viene impiegato in protocolli come TLS e IPsec, nei sistemi di firma digitale come DSA ed ECDSA, oltre ad avere un ruolo cruciale nel funzionamento delle blockchain e dei sistemi di proof-of-work, dove garantisce l'imprevedibilità e la verificabilità delle operazioni.

Capitolo 2

Schnorr: Identificazione e Firma Digitale

In questo capitolo verranno presentati i protocolli di Zero-Knowledge Proof (ZKP), introdotti per la prima volta alla fine degli anni '80 da Shafi Goldwasser, Silvio Micali e Charles Rackoff nel documento *The Knowledge Complexity of Interactive Proof-Systems*[1]. Particolare attenzione sarà dedicata al **protocollo di identificazione di Schnorr** e al corrispondente **schema di firma digitale**.

2.1 Protocolli "Zero Knowledge Proof"

Definizione 2.1.1 (Zero-Knowledge-Proof Protocol). Un protocollo conosciuto nella crittografia come *Zero-Knowledge-Proof Protocol* (indicato con ZK oppure ZKP) rappresenta uno schema di dimostrazione nel quale un soggetto, detto **Prover**, deve convincere un secondo soggetto, detto **Verifier**, che una **dichiarazione** chiamata è vera, senza dover offrire informazioni aggiuntive riguardo il **testimone segreto (Witness)** che la rende vera.

L'intuizione alla base della dimostrazione a conoscenza zero è che è banale convincere del possesso di informazioni sensibili semplicemente rivelandole, mentre risulta più complesso dimostrare il possesso di tali informazioni senza dover rivelarle.

Le dimostrazioni a conoscenza zero possono essere interattive, infatti tra Prover e Verifier si crea una comunicazione nella quale si effettua uno scambio di messaggi secondo un protocollo ben determinato. Esistono anche dimostrazioni a conoscenza zero non interattive, nelle quali il Prover convince il Verifier inviando un solo messaggio.

Al fine di chiarire meglio il funzionamento dei protocolli ZKP, si consideri il seguente esempio astratto.

Si tratta di una storia che racchiude al meglio i principi fondamentali delle dimostrazioni a conoscenza zero. È stato presentato per la prima volta da Jean-Jacques Quisquater in [8].

L'esempio, noto come storia della caverna di Ali Babà, descrive una situazione in cui un Prover (Peggy) deve convincere un Verifier (Victor) di conoscere un segreto (la parola magica che apre una porta nella caverna), senza però rivelarlo direttamente.

In questa storia, Peggy, è in possesso della parola segreta in grado di aprire una porta magica in una caverna. La caverna è a forma di anello con l'ingresso da una parte e, dal lato opposto, la porta magica che impedisce il passaggio da un percorso all'altro come mostrato in Figura 2.1. Victor vuole sapere se Peggy conosce o meno la parola segreta ma Peggy essendo una persona riservata, non vuole rivelare a Victor la parola magica. I percorsi sono etichettati con A (sinistro) e B (destro). Victor aspetta all'entrata mentre Peggy sceglie quale percorso usare senza che Victor lo sappia. Dopodiché, Victor entra nella caverna e grida quale percorso - scelto a caso - deve usare Peggy per fare ritorno all'entrata. A patto che conosca davvero la parola magica, sarebbe facile: aprirebbe la porta, se necessario, per tornare indietro seguendo il percorso desiderato.

Tuttavia, supponendo che Peggy non conosca la parola magica, avrebbe potuto fare ritorno usando il percorso indicato solo se Victor avesse scelto il percorso seguito inizialmente da Peggy. Siccome Victor sceglie casualmente il percorso, lei ha il 50% di probabilità di indovinare correttamente il percorso da seguire per fare ritorno. Se questo processo viene ripetuto, diciamo, 20 volte, allora la probabilità di successo, ovvero di anticipare correttamente la richiesta di Victor, si ridurrebbe a $\frac{1}{2^{20}}$.

In conclusione, se Peggy ripetutamente appare all'entrata seguendo il percorso scelto da Victor, allora si può concludere che Peggy conosce la parola magica.

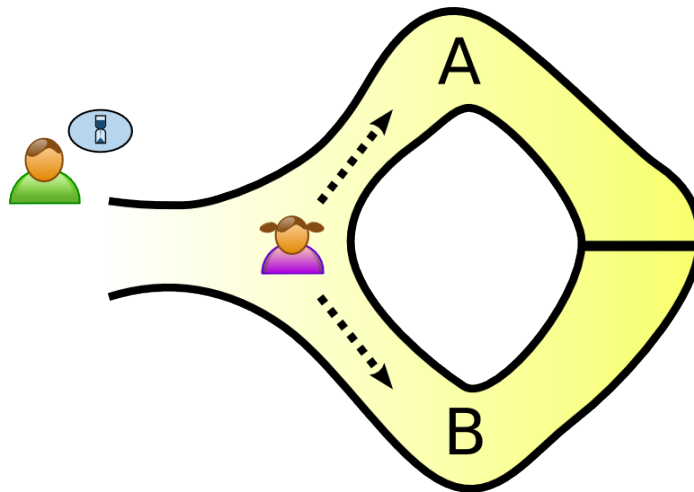


Figura 2.1: Rappresentazione della caverna di Ali Babà.

Ora che è stato chiarito meglio il funzionamento dei protocolli ZKP, si può procedere con elencare le tre proprietà che una dimostrazione ZKP deve soddisfare.

2.1.1 Proprietà

1. **Completezza:** Se l'affermazione è vera, allora un verificatore onesto sarà convinto di questo fatto da un dimostratore onesto.
2. **Correttezza:** Se l'affermazione è falsa, allora nessun dimostratore può imbrogliare un verificatore onesto che il fatto è vero, o meglio la probabilità di riuscire a convincerlo può essere ridotta a piacimento.
3. **Zero Knowledge Proof:** Se l'affermazione è vera, allora nessun verificatore è in grado di acquisire informazioni aggiuntive oltre al fatto che l'affermazione è vera.

La ricerca nell'ambito dei protocolli ZKP ha portato allo sviluppo di sistemi di autenticazione a conoscenza zero, in cui un'entità prova a dimostrare la propria identità ad un'altra entità tramite un'informazione segreta (es. password) senza rivelare tale informazione. Questo tipo di autenticazione è chiamato "Zero Knowledge - Proof of Knowledge" (prova di conoscenza a conoscenza zero).

Per implementare tali protocolli in modo efficiente e sicuro sono stati sviluppati diversi schemi formali. Tra i più studiati e applicati troviamo i cosiddetti Σ -Protocols¹, che costituiscono una classe di protocolli interattivi a tre mosse (commitment, challenge, response) particolarmente adatti alla costruzione di prove di conoscenza a conoscenza zero.

¹Il protocollo d'identificazione di Schnorr è un caso speciale di Sigma Protocol ed è oggetto di studio di questa tesi.

2.2 Protocolli Sigma

In questa sezione si andrà a discutere dei protocolli Σ^2 introdotti per la prima volta da R. Cramer nella sua tesi di dottorato [9]. In particolare verrà data una definizione di cos'è un protocollo Sigma, di come funziona e quali sono le proprietà fondamentali. L'argomento è trattato con maggiori dettagli nel libro D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, Cap. 19.4 [10].

Definizione 2.2.1 (Relazione Effettiva). Una **relazione effettiva** è una relazione binaria $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$, dove $\mathcal{X}$, $\mathcal{Y}$ e $\mathcal{R}$ sono degli insiemi finiti riconoscibili in modo efficiente. Gli elementi di $\mathcal{Y}$ sono detti **affermazioni (statements)**. Se $(x, y) \in \mathcal{R}$, allora x è detto **testimone per (witness for) y** .

Definizione 2.2.2 (Protocollo Sigma). Sia $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ una relazione effettiva. Un **Protocollo Sigma** su $\mathcal{R}$ è una coppia (P, V) .

- P è un protocollo interattivo chiamato **prover**, e prende in input una coppia *testimone-affermazione* $(x, y) \in \mathcal{R}$.
- V è un protocollo interattivo chiamato **verifier**, prende in input un'affermazione $y \in \mathcal{Y}$ (V non è a conoscenza del testimone x), e restituisce in output *accetta* o *rigetta*.
- P e V sono strutturati in modo che un'interazione tra loro avvenga nel seguente modo (vedere Figura 2.2):
 - Per iniziare, P computa un messaggio t , chiamato **commitment**, e lo invia a V .
 - Una volta ricevuto il messaggio t da P , V sceglie casualmente un valore c , chiamato **challenge**, da uno spazio finito $\mathcal{C}$, detto **challenge space**, e lo invia a P .
 - Una volta ricevuta la challenge c da V , P computa z , detto **response**, e lo invia a V .
 - Ricevuto z , V decide se *accettare* o *rigettare* dopo aver calcolato una funzione dipendente dall'affermazione y e dalla conversazione (**conversation**) (t, c, z) .

In particolare, V non effettua ulteriori scelte casuali se non per scegliere c , il resto delle computazioni sono strettamente *deterministiche*.

²Letto *Sigma*, dove la prima parte richiama lo "zig-zag" delle tre mosse tipiche di un protocollo interattivo, mentre il suffisso è un'abbreviazione di "Merlin-Artur".

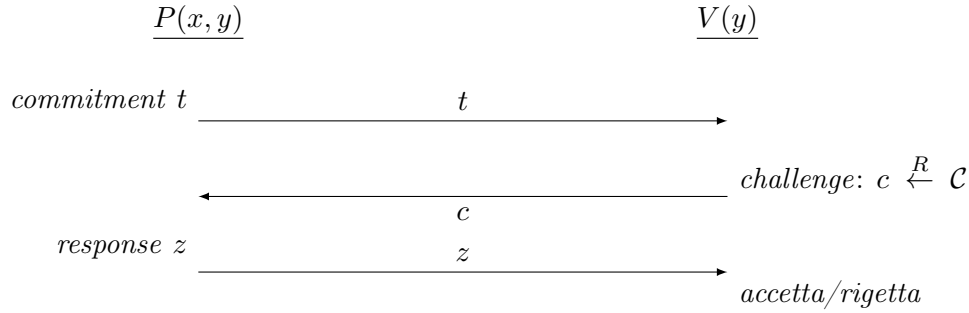


Figura 2.2: Schema di un protocollo Σ interattivo

Come specificato nella Definizione 2.2.2, è richiesto che il verifier calcoli una funzione

$$f : (y, t, c, z) \rightarrow \{\text{accetta}, \text{rigetta}\},$$

dipendente da y e dalla conversazione (t, c, z) . Se l'esito della funzione è *accetta*, allora la conversazione (t, c, z) è detta **accettata per y** .

È chiaro che un'interazione tra un verifier e un prover *onesto* porti sempre ad una conversazione accettata. Conversazioni rigettate si verificano, ad esempio, quando il verifier interagisce con un prover *disonesto* che non rispetta il protocollo concordato.

Inoltre, in molte applicazioni dei protocolli Sigma è richiesto che lo spazio delle challenge $\mathcal{C}$ sia super-poly³.

2.2.1 Proprietà

I protocolli Σ si inseriscono nella categoria dei Zero-Knowledge-Proof e, di conseguenza, rispettano tutte e tre le proprietà fondamentali già elencate nella Sezione 2.1.1. Tuttavia, queste vengono formulate in maniera più specifica, dando origine alle nozioni di **perfect completeness**, **special soundness** e **special honest-verifier zero-knowledge (HVZK)**.

- **Perfect Completeness:** $\forall (x, y) \in \mathcal{R}$, quando $P(x, y)$ e $V(y)$ interagiscono tra loro, $V(y)$ deve *accettare* sempre. Formalmente, abbiamo che:

$$\forall (x, y) \in \mathcal{R}, \Pr[\langle P(x, y), V(y) \rangle = \text{accetta}] = 1$$

dove $\langle P(x, y), V(y) \rangle$ rappresenta l'interazione tra V e P .

³Nella teoria della complessità, quando si parla di **super-poly space** si intende uno spazio di memoria che cresce più velocemente di qualunque polinomio nella dimensione dell'input.

- **Special Soundness:** Sia (P, V) un protocollo Sigma per la relazione $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$. Si dice che (P, V) offre una solidità speciale (**special soundness**) se esiste un algoritmo deterministico efficiente *Ext*, chiamato estrattore di testimoni (**witness extractor**), avente la seguente proprietà: ogni volta che ad *Ext* viene fornito in input un'affermazione $y \in \mathcal{Y}$ e due conversazioni di accettazione (t, c, z) e (t, c', z') , con $c \neq c'$, l'algoritmo restituisce sempre in output $x \in \mathcal{X}$ tale che $(x, y) \in \mathcal{R}$, dove x è il testimone di y .
- **Special HVZK:** Sia (P, V) un protocollo Sigma per la relazione $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ con spazio delle sfide $\mathcal{C}$. Si dice che (P, V) è un verificatore onesto a conoscenza zero (**special HVZK**), se esiste un algoritmo probabilistico efficiente *Sim*, chiamato **simulatore**, che prende in input $(y, c) \in \mathcal{Y} \times \mathcal{C}$ avente le seguenti proprietà:
 1. Per ogni input $(y, c) \in \mathcal{Y} \times \mathcal{C}$, l'algoritmo *Sim* produce in output una coppia (t, z) tale che (t, c, z) è una conversazione accettata per y .
 2. Per ogni $(x, y) \in \mathcal{R}$, se si calcola

$$c \xleftarrow{R} \mathcal{C}, (t, z) \xleftarrow{R} \text{Sim}(y, c),$$

allora (t, c, z) ha la stessa distribuzione di quella di una trascrizione di una conversazione tra $P(x, y)$ e $V(y)$.

In altre parole, questa proprietà dice che, dato un protocollo Sigma (P, V) per $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ e sia la coppia $(x, y) \in \mathcal{R}$, una conversazione tra $P(x, y)$ e $V(y)$ non rivela alcuna informazione aggiuntiva riguardo al testimone x .

A primo impatto, sembrerebbe che la **special soundness** sia una proprietà indesiderabile: Perché vorremmo una proprietà che ci permette di estrarre il testimone x , date due conversazioni accettate? Innanzitutto, è importante osservare che la definizione vale soltanto se le due conversazioni condividono lo stesso *commitment* t mentre le *challenge* c sono diverse. Infatti, possiamo dire che *se qualcuno riesce a rispondere correttamente a due sfide diverse partendo dallo stesso t , allora deve "sapere qualcosa di vero" sul segreto*. Se t fosse diverso, non avrebbe senso dire che "due risposte diverse rivelano il segreto": sarebbero due protocolli indipendenti. Inoltre, poiché t viene scelto in modo casuale dal Prover, è improbabile che un *honest Prover* invii due volte lo stesso t . Quindi, il caso in cui il Verifier ottiene due conversazioni con lo stesso t ma sfide diverse accade solo se il Prover è sotto il controllo dell'estrattore (come avviene nella definizione teorica di soundness).

In particolare, la **special soundness**, ci garantisce:

- **Soundness:** Significa che un Prover che non conosce x non può rispondere correttamente a più di una sfida, perché altrimenti sarebbe possibile estrarre x

dalle sue risposte. Di conseguenza, un truffatore che vuole ingannare il Verifier ha una probabilità di successo al massimo pari a:

$$\frac{1}{|\mathcal{C}|}$$

.

- **Proof of Knowledge:** Consente di costruire un estrattore *Ext*: un algoritmo che, interagendo con un Prover qualunque, può "riavvolgerlo" allo stato precedente alla ricezione della challenge c (mantenendo lo stesso commitment t) e ottenere due conversazioni valide per estrarre il segreto x . Questo serve a formalizzare che il protocollo non dimostra solo che "l'affermazione è vera", ma che il Prover conosce effettivamente il testimone x . Se il Prover fosse uno non onesto, allora non sarebbe possibile generare due conversazioni valide "riavvolgendolo", in quanto per ipotesi, non conosce il testimone x .

2.3 Firme Digitali

Una **firma digitale** è un meccanismo crittografico che permette di garantire l'**autenticità**, l'**integrità** e la **non ripudiabilità** di un documento o di un messaggio elettronico. Nello specifico, si vuole assicurare che un'informazione:

- Provenga davvero dal mittente dichiarato (autenticità).
- Non sia stata alterata durante la trasmissione o l'archiviazione (integrità).
- Non possa essere negata dal mittente (non ripudiabilità).

Dal punto di vista tecnico, la firma digitale si basa sulla crittografia asimmetrica: ogni utente possiede una coppia di chiavi, una privata e una pubblica. La chiave pubblica viene utilizzata per generare la firma sul documento mentre la chiave privata viene usata dai destinatari per verificarne la validità. In pratica, il documento/messaggio non viene firmato direttamente, ma se ne calcola un hash. Questo hash viene poi cifrato con la chiave privata del mittente: il risultato è la firma digitale. Chi riceve il documento potrà ricalcolare l'hash e confrontarlo con quello decifrato usando la chiave pubblica del mittente: se coincidono, la firma è valida.

Le firme digitali sono oggi fondamentali in moltissimi contesti: posta elettronica certificata, documenti legali, contratti online, servizi bancari, e in generale in ogni scenario in cui sia necessario garantire sicurezza e fiducia nello scambio di informazioni.

Di seguito, una definizione formale di firma digitale ripresa da (D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, Cap. 13.1 Pag 528 [10]).

Definizione 2.3.1 (Firma digitale). Un **sistema di firme digitali** $\mathcal{S} = (G, S, V)$ è una tripla di algoritmi efficienti, G, S, V , dove G è chiamato **algoritmo di generazione delle chiavi**, S è chiamato **algoritmo di firma** e, infine, V è detto **algoritmo di verifica**. Dunque, S è usato per *generare* le firme e V è usato per *verificare* le firme. Nello specifico,

- G è un algoritmo *probabilistico* che non richiede nessun input. Fornisce invece in output una coppia (pk, sk) , dove sk è la **chiave segreta di firma** e pk è la **chiave pubblica di verifica**.
- S è anch'esso un algoritmo *probabilistico* che ha il ruolo di computare

$$\sigma \stackrel{R}{\leftarrow} S(sk, m),$$

dove sk è la chiave segreta generata da G ed m è il messaggio. L'algoritmo restituisce in output la **firma** σ .

- V è un algoritmo *deterministico*, tale che

$$\text{accept or reject} \leftarrow V(pk, m, \sigma).$$

- È inoltre richiesto che la firma generata da S sia sempre accettata da V . Ovvero, per ogni coppia (pk, sk) generata da G e per ogni messaggio m , si ha:

$$\Pr[V(pk, m, S(sk, m)) = \text{accept}] = 1.$$

Infine, diremo che i messaggi m risiedono in uno **spazio di messaggi** finito $\mathcal{M}$ e le firme risiedono in uno **spazio di firme** finito Σ . Diremo inoltre che $\mathcal{S} = (G, S, V)$ è definito su $(\mathcal{M}, \Sigma)$.

2.3.1 Euristica di Fiat-Shamir per le Firme

È possibile convertire protocolli Sigma in schemi di firma digitale usando una tecnica sviluppata da Fiat e Shamir conosciuta come **l'euristica di Fiat-Shamir** oppure come la **trasformazione di Fiat-Shamir** [11]. Gli elementi costitutivi sono i seguenti:

- Un protocollo Sigma (P, V) definito per una relazione $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$; assumiamo inoltre che le conversazioni siano della forma (t, c, z) , dove $t \in \mathcal{T}$, $c \in \mathcal{C}$, $z \in \mathcal{Z}$;
- Un algoritmo di generazione delle chiavi G per $\mathcal{R}$;
- Una funzione hash $H : \mathcal{M} \times \mathcal{T} \rightarrow \mathcal{C}$, modellata come un'oracolo casuale; l'insieme $\mathcal{M}$ è lo spazio dei messaggi per lo schema di firme digitali.

Lo schema di firme di Fiat-Shamir derivato da G e dal protocollo Sigma (P, V) funziona nel seguente modo:

- L'algoritmo di generazione delle chiavi è G , dunque, una chiave pubblica è della forma $pk = y$, dove $y \in \mathcal{Y}$ e una chiave segreta è della forma $sk = x$, dove $(x, y) \in \mathcal{R}$.
- Affinché possa firmare un messaggio $m \in \mathcal{M}$ usando una chiave segreta $sk = (x, y)$, l'algoritmo di firma deve funzionare nel seguente modo:
 - Inizializza il prover $P(x, y)$, ottenendo il *commitment* $t \in \mathcal{T}$;
 - Calcola la *challenge* $c \leftarrow H(m, t)$;
 - Infine, immette c nel prover, ottenendo una *response* z e restituendo in output la firma $\sigma := (t, z) \in \mathcal{T} \times \mathcal{Z}$.
- Per verificare la firma $\sigma = (t, z) \in \mathcal{T} \times \mathcal{Z}$ su un messaggio $m \in \mathcal{M}$ usando la chiave pubblica $pk = y$, l'algoritmo di verifica calcola $c \leftarrow H(m, t)$ e verifica che (t, c, z) è una conversazione accettata per y .

2.4 Protocollo d'Identificazione di Schnorr

Il paragrafo che segue è dedicato all'argomento principale di questa tesi: il **protocollo d'identificazione di Schnorr**, ideato da Claus Peter Schnorr alla fine degli anni '80 nel documento *Efficient signature generation by smart cards*[2]. È uno dei più celebri esempi di Sigma protocol basato sul problema del logaritmo discreto. La sua importanza risiede nel fatto che fornisce una base per numerosi schemi di autenticazione moderni e di firme digitali (**firma digitale di Schnorr**).

La struttura del protocollo segue il paradigma **commitment-challenge-response**, tipico dei Sigma protocol, rispettando tutte le proprietà fondamentali: **perfect completeness, special soundness, special HVZK**.

Definizione 2.4.1. Sia $\mathbb{G}$ un gruppo ciclico di ordine primo q con generatore $g \in \mathbb{G}$ e sia $\mathcal{C} \subseteq \mathbb{Z}_q$. Il protocollo d'identificazione di Schnorr è identificato dalla tripla $\mathcal{I}_{sch} = (G, P, V)$, dove:

- G rappresenta l'algoritmo di generazione delle chiavi e funziona nel seguente modo:

$$\alpha \xleftarrow{R} \mathbb{Z}_q, u \leftarrow g^\alpha$$

La chiave di verifica è $vk := u$, mentre la chiave segreta $sk := \alpha$.

- P è il prover inizializzato con $sk = \alpha$ mentre V è il verifier inizializzato con $vk = u$. Il protocollo di comunicazione tra P e V è il seguente, come mostrato in Figura 2.3:

1. P calcola $\alpha_t \xleftarrow{R} \mathbb{Z}_q$, $u_t \leftarrow g^{\alpha_t}$, e invia u_t a V ;
2. V calcola $c \xleftarrow{R} \mathcal{C}$, e lo invia a P ;
3. P calcola $\alpha_z \leftarrow \alpha_t + \alpha c \in \mathbb{Z}_q$, inviandolo poi a V ;
4. V effettua il seguente controllo:

$$g^{\alpha_z} = u_t \cdot u^c$$

Se l'uguaglianza è verificata allora V *accetta*, altrimenti *rigetta*.

L'interazione tra $P(\alpha)$ e $V(u)$ genera una conversazione $(u_t, c, \alpha_z) \in \mathbb{G} \times \mathcal{C} \times \mathbb{Z}_q$. Tale conversazione è detta **di accettazione per u** se il controllo che V effettua è vero, ovvero se vale questa uguaglianza $g^{\alpha_z} = u_t \cdot u^c$. È di facile dimostrazione che qualsiasi interazione tra $P(\alpha)$ e $V(u)$ genera sempre una conversazione di accettazione, infatti, sapendo che $u_t = g^{\alpha_t}$ e $\alpha_z = \alpha_t + \alpha c$, allora

$$g^{\alpha_z} = g^{\alpha_t + \alpha c} = g^{\alpha_t} \cdot g^{\alpha c} = u_t \cdot u^c$$

□

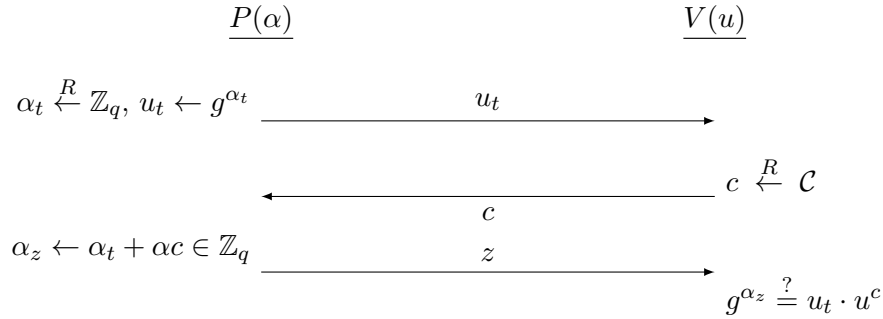


Figura 2.3: Interazione tra P e V nel Protocollo d'identificazione di Schnorr

2.4.1 Proprietà

Una volta chiarito il funzionamento del protocollo d'identificazione di Schnorr, occorre passare alla fase successiva, cioè fornire una dimostrazione della sua sicurezza. Il primo teorema enunciato di seguito dimostra che il protocollo è sicuro contro gli attacchi diretti (enumerazione). La dimostrazione di tale teorema si trova a Pag. 757 del libro Boneh and V. Shoup, *A Graduate Course in Applied Cryptography* [10].

Teorema 2.4.1. *Assumendo che il problema del logaritmo discreto è difficile per il gruppo $\mathbb{G}$ e assumendo inoltre che $N := |\mathcal{C}|$ è super-poly, il protocollo d'identificazione di Schnorr è sicuro contro gli attacchi diretti.*

Dal momento che il protocollo d'identificazione di Schnorr rientra nella classe dei Σ -Protocol, esso soddisfa per costruzione tutte le proprietà fondamentali descritte nella Sezione 2.2.1. Nei paragrafi successivi mostreremo in dettaglio come ciascuna di queste proprietà risulti effettivamente rispettata, iniziando dall'analisi della proprietà di **special HVZK**.

Teorema 2.4.2. *Il protocollo d'identificazione di Schnorr è HVZK.*

Dimostrazione. L'idea è di generare una conversazione simulata (u_t, c, α_z) definendo un simulatore Sim con input $vk = u$ che generi questi messaggi nell'ordine *inverso*. Il simulatore opera nel seguente modo, calcolando:

$$\alpha_z \xleftarrow{R} \mathbb{Z}_q, c \xleftarrow{R} \mathcal{C}, u_t \leftarrow g^{\alpha_z} \cdot u^{-c},$$

producendo quindi in output la conversazione (u_t, c, α_z) .

Rimane da dimostrare che l'output di Sim abbia la stessa distribuzione di una comunicazione reale tra un prover P e un verifier V . L'osservazione chiave, infatti, è che

in un'interazione reale c e α_z sono indipendenti, ovvero c è distribuito uniformemente in $\mathcal{C}$ ed α_z è distribuito uniformemente in $\mathbb{Z}_q$.

Inoltre, dati c e α_z , il prover P calcola il valore u_t determinato univocamente dall'equazione

$$g^{\alpha_z} = u_t \cdot u^c \iff u_t = g^{\alpha_z} \cdot u^{-c}$$

che è esattamente la regola con cui *Sim* calcola u_t . Dunque, la distribuzione di (u_t, c, α_z) prodotta da *Sim* coincide *esattamente* con quella di una conversazione reale tra un prover P e un verifier V . $\square$

Proposizione 2.4.3. *Il protocollo di Schnorr è anche **special HVZK**.*

Dimostrazione. Dato in input $u \in \mathbb{G}$ e $c \in \mathcal{C}$, il simulatore può scegliere $\alpha_z \xleftarrow{R} \mathbb{Z}_q$ e calcolare

$$u_t \leftarrow g^{\alpha_z} \cdot u^{-c}.$$

L'argomento sulla distribuzione rimane identico al caso HVZK e garantisce che la conversazione simulata abbia la stessa distribuzione di quello reale. $\square$

Come corollario del Teorema 2.4.2, segue il seguente teorema:

Teorema 2.4.4. *Se un protocollo d'identificazione $\mathcal{I}$ è sicuro contro gli attacchi diretti ed è HVZK, allora è sicuro contro gli attacchi d'intercettazione.*

Dunque, poiché abbiamo dimostrato che il protocollo d'identificazione di Schnorr è HVZK ed è anche sicuro contro gli attacchi diretti, possiamo concludere, per il Teorema 2.4.4, che è sicuro contro gli attacchi d'intercettazione.

La prossima proprietà che verrà analizzata è la **special soundness**. È facile da verificare che il protocollo d'identificazione di Schnorr rispetta questa proprietà dei protocolli Sigma. L'algoritmo *Ext* (estrattore di testimoni) prende in input $u \in G$, insieme a due conversazioni accettate (u_t, c, α_z) e (u_t, c', α'_z) per u , con $c \neq c'$. Disponendo di queste informazioni possiamo calcolare facilmente $\alpha = \mathsf{D} \log_g u$ (letto: α è il logaritmo discreto in base g di u). Infatti, poiché entrambe le conversazioni sono accettate, abbiamo le seguenti due equazioni:

$$g^{\alpha_z} = u_t \cdot u^c \text{ e } g^{\alpha'_z} = u_t \cdot u^{c'}.$$

Dividendo la prima equazione per la seconda, u_t si cancella e otteniamo:

$$\frac{g^{\alpha_z}}{g^{\alpha'_z}} = \frac{u^c}{u^{c'}} \iff g^{\alpha_z - \alpha'_z} = u^{c - c'} \iff g^{\Delta\alpha} = u^{\Delta c}.$$

Siccome $\Delta c \neq 0$ e poiché q è primo (ordine del gruppo), l'inverso $\frac{1}{\Delta c}$ esiste in $\mathbb{Z}_q$. Possiamo dunque elevare entrambe le parti a potenza $\frac{1}{\Delta c}$, ottenendo:

$$g^{\frac{\Delta\alpha}{\Delta c}} = u.$$

Pertanto, possiamo calcolare in modo efficiente $\alpha = \mathsf{D} \log_g u = \frac{\Delta\alpha}{\Delta c}$. $\square$

2.4.2 Schema di Firma Digitale di Schnorr

In questa sezione verrà presentato come è possibile trasformare il protocollo d'identificazione di Schnorr in uno schema di firme seguendo la trasformazione di Fiat-Shamir discussa nella Sezione 2.3.1.

Dato che il protocollo d'identificazione di Schnorr è definito su un gruppo ciclico $\mathbb{G}$ di ordine q primo con generatore $g \in \mathbb{G}$, e che lo spazio delle challenge $\mathcal{C} \in \mathbb{Z}_q$, resta da determinare la funzione hash

$$H : \mathcal{M} \times \mathbb{G} \rightarrow \mathcal{C},$$

dove $\mathcal{M}$ rappresenta lo spazio dei messaggi. La firma di un messaggio $m \in \mathcal{M}$ sarà la coppia (u_t, α_z) dove (u_t, c, α_z) è una conversazione accettata per la chiave di verifica u , esattamente come avviene nel protocollo d'identificazione di Schnorr, mentre la challenge c è calcolata come $c \leftarrow H(m, u_t)$. Nel dettaglio, lo schema di firme di Schnorr è $S_{sch} = (G, S, V)$, dove:

- L'algoritmo di generazione delle chiavi G funziona nel seguente modo:

$$\alpha \xleftarrow{R} \mathbb{Z}_q, u \leftarrow g^\alpha.$$

La chiave pubblica è $pk := u$ mentre la chiave segreta è $sk := \alpha$.

- Per firmare un messaggio $m \in \mathcal{M}$ usando la chiave segreta $sk = \alpha$, l'algoritmo di firma funziona nel seguente modo:

$$S(sk, m) := \alpha_t \xleftarrow{R} \mathbb{Z}_q, u_t \leftarrow g^{\alpha_t}, c \leftarrow H(m, u_t), \alpha_z \leftarrow \alpha_t + \alpha c$$

restituendo infine in output la firma $\sigma := (u_t, \alpha_z)$.

- Infine, per verificare la firma $\sigma = (u_t, \alpha_z)$ su un messaggio $m \in \mathcal{M}$ usando la chiave pubblica $pk = u$, l'algoritmo di verifica V calcola $c \leftarrow H(m, u_t)$ e verifica se l'uguaglianza $g^{\alpha_z} = u_t \cdot u^c$ è vera.

Capitolo 3

Progetto Sperimentale - SchnorrAuthApp

Questo capitolo è dedicato esclusivamente allo sviluppo di un sistema di autenticazione basato sul protocollo d'identificazione di Schnorr. L'obiettivo di questo progetto è di fornire un sistema di autenticazione sicuro che permetta all'utente di autenticarsi senza l'utilizzo di una password, infatti, come vedremo meglio nei paragrafi successivi, l'utente avrà bisogno solamente di uno username. Oltre a questo, ciascun utente avrà la possibilità di associare più dispositivi ad un unico account, acquisendo, in tal modo, la capacità di autenticarsi in seguito senza password dal nuovo dispositivo associato.

L'intero sistema è centralizzato in un server Python (Python Software Foundation, 2025 *Python: A dynamic, open source programming language* [12]) e un database MongoDB (MongoDB Inc. 2025 *Mongodb: The developer data platform* [13]) usato per memorizzare le informazioni riguardo l'account di ciascun utente. A scopi didattici, è stato poi sviluppato un client Python, eseguibile da riga di comando, compatibile solo con sistemi Linux, che permette di collegarsi al server mediante socket TCP (W. Eddy, *Transmission Control Protocol (TCP)* RFC 9293 [14]) e stabilire una connessione client - server. Una volta stabilita la connessione, il client offre la possibilità di registrarsi, autenticarsi e, se necessario, di accoppiare un nuovo dispositivo.

Infine, è stata sviluppata un'applicazione client Android usando il framework Flutter (Google, *Flutter: Build apps for any screen*, 2025 [15]).

Il codice del server e del client Python che implementano il funzionamento del protocollo d'identificazione di Schnorr è possibile trovarlo nel repository al seguente link: <https://github.com/IonutZbir/ServerClientSchnorr>.

Il codice sorgente dell'applicazione Flutter è possibile trovarlo nel repository al seguente link: <https://gitlab.com/IonutZbir/SchnorrAuthApp>.

3.1 Analisi Architetture

Prima di procedere con la spiegazione dettagliata del funzionamento del sistema, è necessario fare le opportune analisi delle scelte architetture, per capirne al meglio l'implementazione.

Al fine di sviluppare il sistema di autenticazione mediante protocollo d'identificazione di Schnorr, è stato definito il protocollo di comunicazione (**SAP - Schnorr-based Authentication Protocol**, analizzato successivamente nella Sezione 3.2) basato sul modello classico *client-server*. Poiché è un sistema interattivo, SAP è **stateful**, ovvero è un protocollo di comunicazione che mantiene lo stato della sessione tra le richieste.

3.1.1 Scelta e Definizione del Database

Come menzionato prima, l'architettura sulla quale è basato il sistema è di tipo *client-server*, dove il client ha il ruolo di *prover* mentre il server ha il ruolo di *verifier*. Il database scelto è MongoDB, un database non relazionale. Si è deciso quindi di optare per MongoDB anziché un database relazionale in quanto:

- **Praticità:** è più pratico in quanto i dati da memorizzare non sono molto complessi, infatti, le entità sono abbastanza elementari. Inoltre, sono documenti molto naturali da rappresentare in formato JSON. MongoDB, in questo senso, è perfetto, perché permette di salvare e recuperare velocemente questi documenti senza dover progettare un intero schema relazionale con tabelle, vincoli e join.
- **Flessibilità:** un altro punto è la flessibilità dello schema. Durante lo sviluppo del sistema di autenticazione la struttura dei dati è cambiata innumerevoli volte e, con MongoDB, questo è possibile farlo senza dover modificare una o più migrazioni SQL.
- **Semplicità:** infine, c'è l'aspetto della semplicità di integrazione con il codice. Infatti, usando Python, lavorare con dizionari e oggetti che finiscono direttamente in un documento MongoDB è molto lineare in quanto la struttura dei dizionari Python è molto simile ai documenti JSON.

La comunicazione tra Python e MongoDB è possibile tramite la libreria **beanie**[16], un *Object-Document Mapper* (ODM) asincrono per MongoDB. Tale libreria si occupa di astrarre la gestione diretta dei documenti, permettendo di interagire con essi come se fossero oggetti Python, semplificando notevolmente le operazioni di lettura, scrittura e aggiornamento dei dati. I modelli di dati sono basati su **Pydantic**[17] un potente framework per la definizione e la validazione dei modelli, che garantisce coerenza e correttezza dei dati memorizzati nel database.

Nel progetto in esame, i modelli dei documenti sono stati dichiarati all'interno del file `server/models.py`. In questo file vengono descritte le strutture dei dati che rappresentano le diverse entità del sistema.

```
class PublicKey(Document):
    pk: str
    hash_pk: str
    device_name: str
    logged: bool

class User(Document):
    username: str
    public_keys: List[Link[PublicKey]]
    created_at: datetime = datetime.now()

class HashedUser(Document):
    hash_pk: str
    user: Link[User]

class Pairing(Document):
    hash_pk: str
    pk: Link[PublicKey]
    created_at: datetime
    expiry: datetime

    @property
    def is_expired(self):
        return datetime.now() > self.expiry
```

Codice 1: Definizione dei modelli all'interno del file `server/models.py`

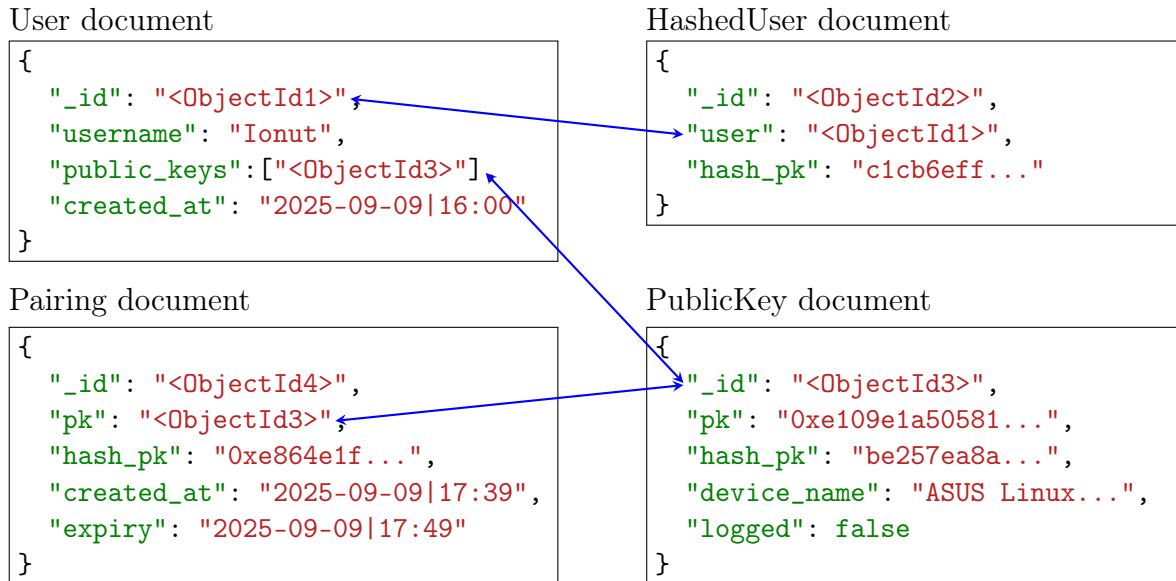


Figura 3.1: Database Schema

Come mostrato in Figura 3.1 e come definito nel Codice 1, lo schema del database consiste di 3 documenti:

- **User:** è il documento principale nel quale sono memorizzate le informazioni di ciascun utente, come lo username e la lista delle chiavi pubbliche associate ad esso.
- **HashedUser:** rappresenta una mappatura `hash_pk → user`, utile per identificare in modo rapido l'utente che sta provando ad autenticarsi usando l'hash della sua chiave pubblica. Infatti, al momento dell'autenticazione, il client invierà oltre allo username anche l'hash della chiave pubblica per identificare facilmente e in modo univoco l'utente che prova ad autenticarsi.
- **PublicKey:** contiene le informazioni di ciascuna chiave pubblica registrata, il dispositivo sul quale è memorizzata, il suo hash e infine un campo booleano `logged`.
- **Pairing:** viene aggiunto un nuovo record quando un dispositivo *slave* chiede di essere abbinato ad un account. È un documento di appoggio che permette di eseguire l'accoppiamento dei dispositivi.

3.1.2 Calcolo dell'Hash delle Chiavi Pubbliche

L'hash viene calcolato usando un algoritmo crittografico di hash (ad esempio SHA-256 discusso nella Sezione 1.4), che restituisce una stringa di lunghezza fissa. Questo valore viene poi memorizzato nel campo `hash_pk` e utilizzato come indice per risalire rapidamente all'utente associato, senza dover trasmettere direttamente la chiave pubblica completa ogni volta che l'utente si prova ad autenticare. Infatti, in questo modo la chiave pubblica viene comunicata al server una sola volta in fase di registrazione.

```
import hashlib
def hash_public_key_SHA256(public_key: int) -> 'hashlib.Hash':
    pk_len_bits = public_key.bit_length()
    pk_bytes = public_key.to_bytes((pk_len_bits + 7) // 8)
    return hashlib.sha256(pk_bytes)
```

Codice 2: Calcolo dell'hash della chiave pubblica in Python

3.1.3 Salvataggio della Chiave Segreta in Locale

Un' ulteriore scelta architetturale è stata quella di salvare su file locale la chiave privata di ciascun account registrato dal dispositivo. Nello specifico, andremo a discutere quale strategia è stata applicata per salvare le chiavi private su dispositivi Linux e perché tale strategia è sicura.

Quando l'utente si registra da un dispositivo Linux, la chiave viene salvata all'interno di un file `user_privkey.pem` nella directory `/home/user/.schnorr`. Inoltre, il client ha la possibilità di impostare una password, la quale verrà poi usata per derivare la chiave di cifratura dell' algoritmo di cifratura a blocchi AESGCM discusso nella Sezione 1.3. Se l'utente non imposta una password, allora la chiave viene salvata in chiaro senza applicare nessun algoritmo di cifratura.

Nell'implementazione Python del client, il modulo che si occupa di caricare e salvare la chiave su file locale si trova in `client/utlis.key_manager.py`. Un'implementazione simile a quella reale si trova in Appendice, nella Sezione A.1.

3.1.4 SchnorrProver e SchnorrVerifier

Come discusso all'inizio del capitolo, il client assume il ruolo di *prover*, mentre il server ricopre quello di *verifier*. In particolare, ogni client istanzia un oggetto della classe `SchnorrProver` e, per ciascuna fase del protocollo, utilizza la chiave privata α , la coppia di chiavi temporanee (α_t, u_t) — dove u_t rappresenta il *commitment* inviato al *verifier* — e la risposta α_z , calcolata dopo aver ricevuto la *challenge*. Sul

lato server, per ogni client in ingresso viene creata una task¹ insieme a un'istanza della classe `SchnorrVerifier`. Le due classi hanno il ruolo di centralizzare le operazioni aritmetiche necessarie per il corretto funzionamento del protocollo d'identificazione e per la firma digitale di Schnorr. In Appendice, nella Sezione A.2 è mostrata un'implementazione semplificata delle due classi insieme ad un esempio di funzionamento.

3.2 Funzionamento del Protocollo SAP

Il funzionamento di SAP è diviso in 3 fasi fondamentali: la prima consiste in una fase di *handshake*, nella quale client e server concordano i parametri necessari alla comunicazione, la seconda è la fase di registrazione nella quale il client si registra presso il server e la terza è la fase di autenticazione che segue nei minimi dettagli il protocollo di Schnorr.

3.2.1 Fase di Handshake

Una volta stabilita la connessione tra client e server ha luogo una fase di *handshake*. In questa fase, i due attori concordano i parametri necessari per l'autenticazione mediante protocollo d'identificazione di Schnorr, ossia, il gruppo matematico da utilizzare per i calcoli crittografici. Questo scambio iniziale consente di sincronizzare client e server, garantendo che entrambi operino con gli stessi valori di riferimento prima di procedere con le successive fasi del protocollo.

Sia client che server dispongono di una lista di gruppi crittografici con i quali sono abilitati a lavorare. Il server, al momento della connessione di un client, invia una lista dei gruppi crittografici a cui è abilitato per i calcoli matematici. Il client verifica, poi, se dispone dei gruppi ricevuti dal server e sceglie il primo disponibile. Se non dispone di nessun gruppo, invia un messaggio al server comunicando che la sincronizzazione non può avvenire e chiude la connessione. Per la scelta dei gruppi crittografici da utilizzare è stato adoperato il seguente standard *RFC 3526* [3], nel quale è presente una lista di gruppi crittografici standardizzati.

La fase di *handshake* consiste nei seguenti passaggi, come mostrato in Figura 3.2.

1. Il client invia al server un messaggio `{"type": "HANDSHAKE_REQ"}`;
2. Il server risponde, poi, al client con un messaggio del tipo

¹Il server utilizza il modulo `asyncio` per la gestione concorrente, basata su *task* asincroni piuttosto che su *thread* nativi.

```
{  
  "type": "HANDSHAKE_RES",  
  "crypto_groups": ["modp-2048", "modp-4096"]  
}
```

3. Il client verifica se è abilitato ad usare almeno uno dei gruppi ricevuti dal server e ne sceglie uno, ad esempio il primo disponibile, e lo invia al server come

```
{  
  "type": "HANDSHAKE_OK",  
  "group_id": "modp-4096"  
}
```

Se, invece, non è abilitato ad usare nessuno dei gruppi inviati dal server, allora risponde con `{"type": "HANDSHAKE_NOK"}` e chiude la connessione.

4. Il server riceve il gruppo crittografico dal client e lo imposta per eseguire tutti i calcoli matematici necessari per il corretto funzionamento del protocollo SAP.

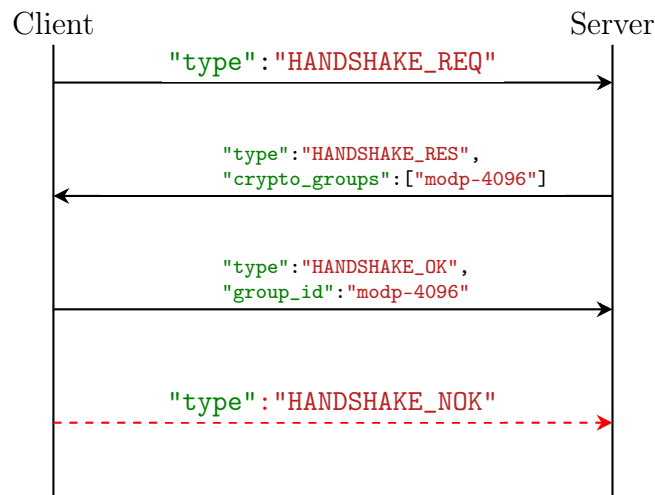


Figura 3.2: Interazione tra client e server durante la fase di handshake

3.2.2 Fase di Registrazione/Autenticazione

Una volta impostati i parametri stabiliti nella fase di handshake, client e server sono ora pronti per poter comunicare mediante **SAP**. Adesso l'utente si può autenticare/-registrare sul server digitando uno username, qualsiasi esso sia. Uno degli obiettivi principali, oltre alla sicurezza di questo sistema d'autenticazione client-server, è quello di mascherare all'utente finale la fase di *registrazione* al server. L'utente non deve essere a conoscenza del fatto che si stia registrando al server ma deve sapere soltanto che si sta autenticando. Infatti, l'unica opzione di cui dispone l'utente è quella di accedere mediante username. Spetta, quindi, al sistema determinare se procedere o meno con la registrazione dell'account.

La fase di autenticazione prevede che l'utente inserisca il proprio username, dopodiché il client software proverà a caricare dal disco² la chiave privata α . Poiché uno degli obiettivi principali di questo sistema di autenticazione è di nascondere all'utente finale la fase di registrazione, questa avviene quando il file contenente la chiave privata non è presente sul disco e, in tal caso, si procede con la registrazione del nuovo account.

In seguito, verrà mostrato un esempio di funzionamento delle fasi registrazione e autenticazione al fine di comprenderlo al meglio. Supponiamo che l'utente *Ionut* si voglia autenticare dal suo dispositivo portatile **LENOVO**, il client prova a leggere dal disco la chiave privata di *Ionut*, allora:

- Se il file con la chiave non esiste, allora significa che *Ionut* non è registrato, o meglio, non è presente la sua chiave in locale, in quanto sul server potrebbe già essere presente un account registrato con lo username *Ionut*. Infatti, il server per distinguere due utenti con lo stesso username identifica in modo univoco ciascun utente mediante un identificatore univoco generato in automatico da MongoDB. In alternativa, si potrebbe utilizzare uno `uuid` generato al momento della registrazione, ma per semplicità è stato scelto di usare l'object id di Mongo. A questo punto, il client genera la sua coppia di chiavi, $(\alpha, u := g^\alpha)$ richiamando il metodo `SchnorrProver.genKeys()`, come discusso nella Sezione 3.1.4. La chiave privata α verrà memorizzata su file locale, mentre la chiave pubblica u verrà inviata, in seguito, al server insieme al nome del dispositivo da cui si è effettuata la registrazione, in un messaggio di tipo:

```
{
  "type": "REGISTRATION_REQ",
  "username": "Ionut",
  "public_key": u,
  "device": "LENOVO"
}
```

²La modalità con il quale il sistema cifra e salva poi su disco la chiave privata dell'utente viene discussa nella Sezione A.1

Ricevuta la richiesta dal client, il server memorizza all'interno del database il nuovo utente, creando un nuovo oggetto `User` e `PublicKey`. Infine, crea un nuovo oggetto `HashedUser` mappando in questo modo l'hash della chiave pubblica al nuovo utente ($\text{SHA256}(u) \rightarrow \text{username}$), facilitando l'operazione di ricerca durante la fase di autenticazione³. A questo punto, il server risponde al client comunicando l'avvenuta registrazione con successo dell'account.

```
{"type": "REGISTERED", "username": "Ionut"}
```

- Se, invece, il file con la chiave privata esiste, allora il client procede con la fase di autenticazione. Dunque, genera le chiavi temporanee ($\alpha_t, u_t := g^{\alpha_t}$), ricordando che u_t rappresenta il *commitment*, e invia quest'ultima al server insieme all'hash della sua vera chiave pubblica u .

```
{
  "type": "AUTH_COMMITMENT",
  "username": "Ionut",
  "public_key_temp":  $u_t$ ,
  "pk_hash":  $\text{SHA256}(u)$ 
}
```

Ricevuto il messaggio contenente il commitment, il server usa l'hash della chiave pubblica per interrogare il database, così da individuare l'utente corrispondente. Se l'utente esiste, allora, procede con la generazione della challenge c e la invia al client.

```
{
  "type": "AUTH_CHALLENGE",
  "challenge":  $c$ 
}
```

Ricevuta la challenge, il client calcola la *response* $\alpha_z = \alpha_t + \alpha c$, comunicandola al server.

```
{
  "type": "AUTH_RESPONSE",
  "response":  $\alpha_z$ 
}
```

A questo punto, il server (verifier), è in possesso di tutti i parametri necessari per passare alla verifica del client (prover). Per ogni chiave pubblica registrata

³Per maggiori dettagli, vedere Sezione 3.1.1

presso l'utente Ionut⁴, richiama il metodo `SchnorrVerifier.check(res)`. Se dà esito positivo, allora comunica al client che è stato autenticato con successo (`{"type": "AUTH_ACCEPTED"}`), altrimenti, se non viene autenticato viene inviato un messaggio (`{"type": "AUTH_REJECTED"}`).

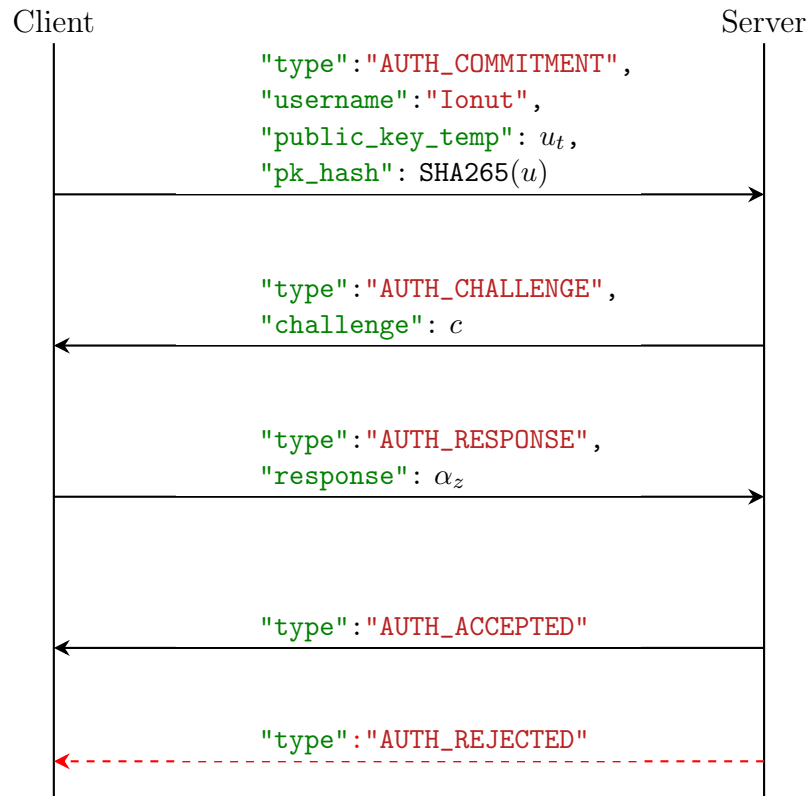


Figura 3.3: Interazione tra client e server durante la fase di autenticazione

⁴Ad ogni utente possono corrispondere una o più chiavi pubbliche, in quanto ciascun utente ha la possibilità di associare più dispositivi o addirittura unire più account.

3.2.3 Processo di Accoppiamento di un Dispositivo

Il principale svantaggio di un sistema di autenticazione *passwordless* risiede proprio nell'assenza della password, in quanto la password/chiave privata è device/client-dependent. Di conseguenza, se l'utente dovesse perdere la propria chiave privata (ad esempio a seguito di una formattazione o dell'eliminazione accidentale del file che la contiene) il suo account risulterebbe inaccessibile, non potendo più autenticarsi al server. Una soluzione valida per mitigare questo problema è di offrire la possibilità agli utenti di poter associare più dispositivi allo stesso account, in questo modo, anche in caso di perdita della chiave privata su un dispositivo, sarebbe comunque possibile accedere tramite un altro.

L'idea generale del meccanismo di associazione è quella di impiegare la firma digitale di Schnorr⁵ nella fase di conferma. Quando un utente intende associare un nuovo dispositivo (*slave*), la procedura di accoppiamento deve essere confermata tramite un dispositivo già autenticato (*master*).

In pratica, al momento della generazione della coppia di chiavi da parte dello slave, la chiave pubblica viene codificata in un `qr_code` e mostrata a schermo. Se il dispositivo master è mobile, l'utente può semplicemente scansionare il `qr_code`, inviando al server la chiave pubblica insieme alla relativa firma. La situazione si complica invece quando il dispositivo master è un desktop: in assenza di un lettore di `qr_code`, l'utente è costretto a inserire manualmente la chiave pubblica visualizzata, con un impatto negativo sulla user experience, poiché digitare una chiave pubblica risulta scomodo ed soggetto ad errori.

Per migliorare la user experience anche in questo scenario, si è reso necessario definire un formato alternativo per la stringa da inserire: le opzioni considerate sono state un token generato casualmente oppure una sequenza di parole in italiano.

La soluzione preferita è quella basata su una sequenza di parole, implementata mediante lo standard BIP-39 (*Bitcoin Improvement Proposal*)[18]. Questo BIP definisce l'utilizzo di un codice mnemonico, ossia una frase composta da parole di facile memorizzazione, originariamente pensata per la generazione di *wallet* deterministici. Nel nostro caso, non viene generata entropia, bensì si utilizzano direttamente i bit della chiave pubblica. Tali bit vengono suddivisi in blocchi da 11, ciascuno dei quali rappresenta un numero intero compreso tra 0 e 2047, usato come indice all'interno di una lista predefinita di 2048 parole.

Tuttavia, questo approccio non risulta pratico: la codifica di una chiave pubblica, la cui lunghezza può variare da 1536 a 4096 bit, a seconda del gruppo crittografico utilizzato, genererebbe infatti una frase mnemonica composta da circa 100–300 parole, compromettendo seriamente la *user experience*. Per ovviare a questo problema, si è scelto di adottare un codice mnemonico molto più compatto, composto da una decina

⁵Per maggiori dettagli consultare la Sezione 2.4.2.

di parole, facilmente memorizzabile e semplice da comunicare anche verbalmente. Inoltre, si è previsto di integrare un sistema di autocompletamento, sfruttando il fatto che la lista di parole definita dallo standard BIP-39 è ordinata alfabeticamente. Ciò consente un recupero delle parole più efficiente, ad esempio tramite ricerca binaria invece che lineare, e permette anche l'utilizzo di strutture dati come i **trie** per ottenere una migliore compressione.

Pertanto, anziché codificare direttamente l'intera chiave pubblica, essa viene prima sottoposta alla funzione di hash **SHA-256** e successivamente ridotta a 160 bit tramite l'algoritmo **RIPEMD-160**. L'intero risultato di 160 bit viene quindi utilizzato per generare la frase mnemonica, che risulta composta da circa 15 parole. Questa scelta consente di ottenere un buon compromesso tra sicurezza e usabilità, mantenendo la frase sufficientemente compatta da poter essere ricordata e comunicata con facilità, senza rinunciare a un adeguato livello di protezione crittografica.

```
from Crypto.Hash import RIPEMD160
hash_pk = hash_public_key_SHA256(pk)
ripemd160_pk = RIPEMD160.new()
ripemd160_pk.update(hash_pk.digest())
```

Codice 3: Hash della chiave pubblica con RIPEMD-160

Con questo meccanismo, il dispositivo slave invia al server la propria chiave pubblica. Il dispositivo master, invece, ricava il medesimo prefisso a partire dalla sequenza di parole e lo trasmette al server accompagnato dalla firma. Una volta ricevuto il messaggio dal master, il server ne verifica la firma e, se valida, associa la chiave pubblica all'account dell'utente, completando l'aggiunta del dispositivo slave.

In particolare, quando il server riceve la richiesta di associazione dal dispositivo slave, crea un nuovo oggetto **Pairing**, contenente a sua volta un oggetto **PublicKey**, insieme all'hash calcolato con **RIPEMD-160** e lo memorizza nel database.

Successivamente, alla ricezione del messaggio di conferma dal master, esegue la verifica della firma tramite il metodo **SchnorrVerifier.verify_sign(sign, message)**, dove **sign** è la firma $\sigma = (u_t, \alpha_z)$ inviata dal client master e **message** rappresenta l'hash della chiave pubblica. Se la verifica fallisce, il server invia a entrambi i client un messaggio **{"type": "ASSOC_REJECTED"}**. In caso contrario, interroga il database utilizzando l'hash per recuperare l'oggetto **Pairing** precedentemente generato e ottenere la chiave pubblica del dispositivo slave. Se l'oggetto esiste, aggiorna l'account dell'utente inserendo la nuova chiave pubblica.

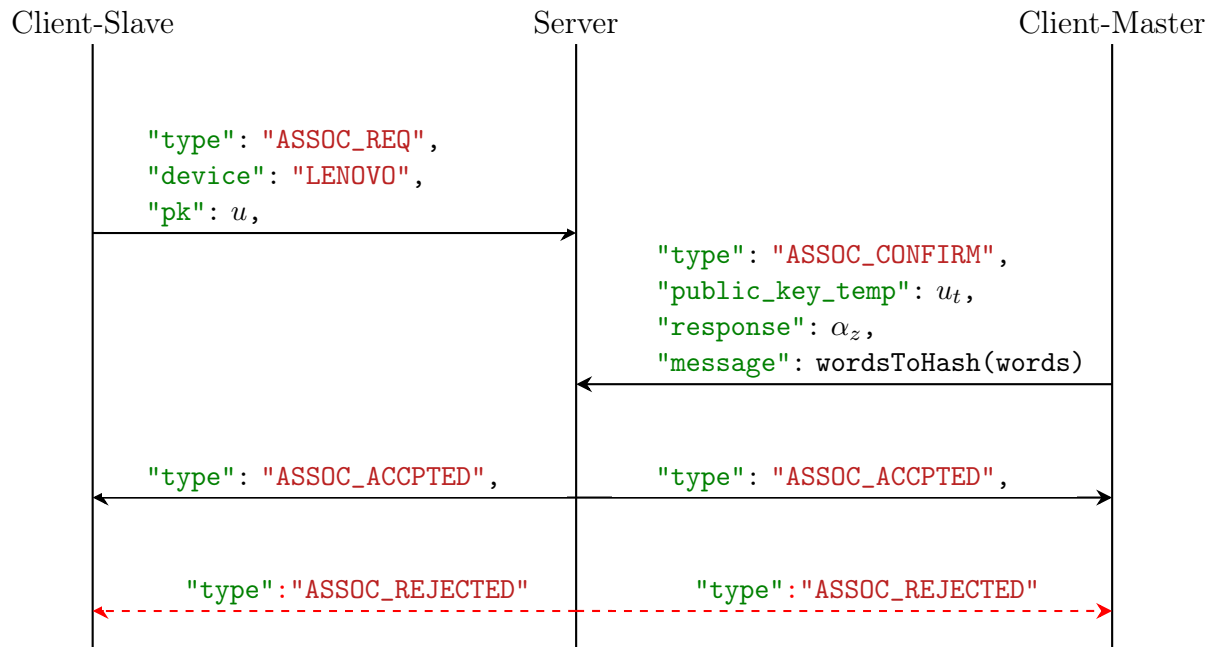


Figura 3.4: Interazione tra slave, master e server durante il processo di accoppiamento.

3.3 Applicazione Client Flutter

Nelle sezioni precedenti è stato illustrato il funzionamento del protocollo **SAP**, con particolare attenzione allo scambio di messaggi tra client (prover) e server (verifier). Finora il client considerato era sviluppato in Python ed eseguibile da linea di comando esclusivamente su sistemi Linux. Per rendere l'utilizzo più accessibile e intuitivo, si è scelto di realizzare un'applicazione con interfaccia grafica, sviluppata in Flutter [15].

La scelta di Flutter è motivata soprattutto dal fatto che si tratta di un framework multiplatforma: consente infatti di scrivere il codice in un unico linguaggio (Dart [19]) e successivamente generare l'applicazione per diversi ambienti, come Android per dispositivi mobile e Linux, Windows per dispositivi desktop.

Dal punto di vista dell'esperienza utente, l'app è stata progettata per risultare semplice e immediata: l'accesso avviene inserendo unicamente lo username, dopodiché l'interfaccia mostra lo stato della connessione e l'elenco dei dispositivi già associati all'account. L'aggiunta di un nuovo dispositivo avviene in maniera differente a seconda della piattaforma utilizzata come master: se si tratta di un dispositivo mobile, è possibile effettuare l'accoppiamento tramite la scansione del QR code generato dal dispositivo slave; se invece il master è un computer desktop, è stato previsto un sistema di autocompletamento che semplifica e velocizza la procedura di associazione.

3.3.1 F-Droid

Un aspetto importante nello sviluppo dell'app è stato quello di garantirne la piena compatibilità con le linee guida di **F-Droid** [20], il principale repository di applicazioni *Free and Open Source Software (FOSS)* per dispositivi Android. F-Droid si distingue dagli store tradizionali perché distribuisce esclusivamente software libero, privo di componenti proprietarie, tracciamenti occulti o dipendenze che possano compromettere la trasparenza e la libertà dell'utente. Ogni applicazione pubblicata deve rispettare criteri rigorosi, tra cui la disponibilità del codice sorgente, la possibilità di verificarne e ricompilarne in maniera indipendente i binari, e l'assenza di meccanismi pubblicitari o sistemi di tracking.

L'applicazione sviluppata soddisfa pienamente questi requisiti sotto diversi punti di vista. Anzitutto, l'intero codice sorgente è reso disponibile pubblicamente su **GitLab**, piattaforma web open source per la gestione collaborativa di repository **Git**. Ciò consente non solo la condivisione e il versionamento del codice, ma anche la possibilità per chiunque di ispezionarlo e verificarne il corretto funzionamento⁶. Inoltre, tutte le librerie e i pacchetti impiegati sono anch'essi open source e reperibili in repository pubblici, garantendo così la totale trasparenza dell'intera catena di dipendenze. Particolare attenzione è stata rivolta anche alla gestione dei dati dell'utente: non ven-

⁶Repository GitLab: <https://gitlab.com/IonutZbir/SchnorrAuthApp>

gono raccolte informazioni personali⁷ né inviati metadati a terze parti, in linea con il principio di minimizzazione dei dati promosso da F-Droid. Infine, l'assenza di sistemi pubblicitari o meccanismi di profilazione assicura che l'app rimanga uno strumento sicuro e rispettoso della privacy.

Grazie a queste caratteristiche, l'app non solo soddisfa i requisiti minimi per una distribuzione su F-Droid, ma incarna a pieno i valori del software libero: trasparenza, sicurezza, rispetto della privacy e possibilità di revisione comunitaria.

In aggiunta, la struttura della repository deve rispettare i requisiti stabiliti da F-Droid, comprendendo i metadati necessari all'applicazione. A tal fine, si è deciso di adottare l'organizzazione standard compatibile con *fastlane*, così da garantire una gestione coerente e facilmente integrabile nel processo di distribuzione⁸.

3.3.2 Livelli di Sicurezza

Dal momento che la chiave segreta viene conservata localmente sul dispositivo, per assicurarne la protezione è stata prevista la sua cifratura (per ulteriori dettagli si rimanda alla Sezione A.1). L'accesso alla chiave avviene quindi tramite l'inserimento di una password (PIN); sui dispositivi mobili, in alternativa, è possibile sbloccarla utilizzando l'autenticazione biometrica tramite impronta digitale.

In particolare, sono stati implementati due livelli di sicurezza:

Livello 1 di Sicurezza – L'utente può decidere se autenticarsi tramite PIN oppure tramite impronta digitale. Nel caso in cui venga scelta l'opzione biometrica, al primo avvio dell'app viene generata in automatico una password di 32 caratteri, che viene memorizzata localmente sfruttando la libreria Flutter `flutter_secure_storage`. L'impronta digitale ha quindi il solo scopo di sbloccare ogni tal volta la password utilizzata per la cifratura della chiave segreta.

Livello 2 di Sicurezza - Poiché l'impiego di un PIN scelto direttamente dall'utente (ad esempio sequenze banali come 1234) non garantisce un livello di sicurezza adeguato, l'applicazione mette a disposizione un secondo livello di protezione. In questo caso viene utilizzata una password di 32 caratteri generata in modo casuale, analoga a quella prevista nel livello 1 quando si adotta l'autenticazione biometrica. Pertanto, se l'utente decide di inserire manualmente un PIN (nel caso in cui il dispositivo è desktop oppure il dispositivo mobile

⁷Infatti, l'unico dato dell'utente che viene conservato sul server è lo *username*; di conseguenza, ciascun utente ha la piena libertà di scegliere il nome con cui desidera essere identificato all'interno dell'applicazione.

⁸Per maggiori dettagli, visitare: https://f-droid.org/en/docs/All_About_Descriptions_Graphics_and_Screenshots/

non dispone di autenticazione biometrica), ciò che in realtà deve digitare è la password generata dall'app. Il principale svantaggio di questa soluzione risiede nel fatto che l'utente dovrebbe ricordare una stringa lunga e complessa; per superare tale problema si è adottato lo stesso approccio previsto per la codifica della chiave privata durante la fase di associazione. In particolare, la password viene convertita in una sequenza mnemonica, che viene mostrata all'utente e che egli è invitato a conservare con cura, poiché rappresenta la codifica leggibile della password generata.

3.3.3 Interfaccia Grafica

Per completare la descrizione dell'applicazione sviluppata, si ritiene utile mostrare alcune schermate rappresentative dell'interfaccia grafica. Queste immagini hanno lo scopo di fornire al lettore una visione concreta delle funzionalità principali e del flusso d'uso, mettendo in evidenza come le scelte progettuali discusse nei paragrafi precedenti siano state tradotte in una GUI semplice e intuitiva. Le figure che seguono illustrano i passaggi fondamentali: dalla fase di registrazione/accesso, alla visualizzazione dello stato della connessione e dei dispositivi associati, fino alla procedure di accoppiamento e conferma.

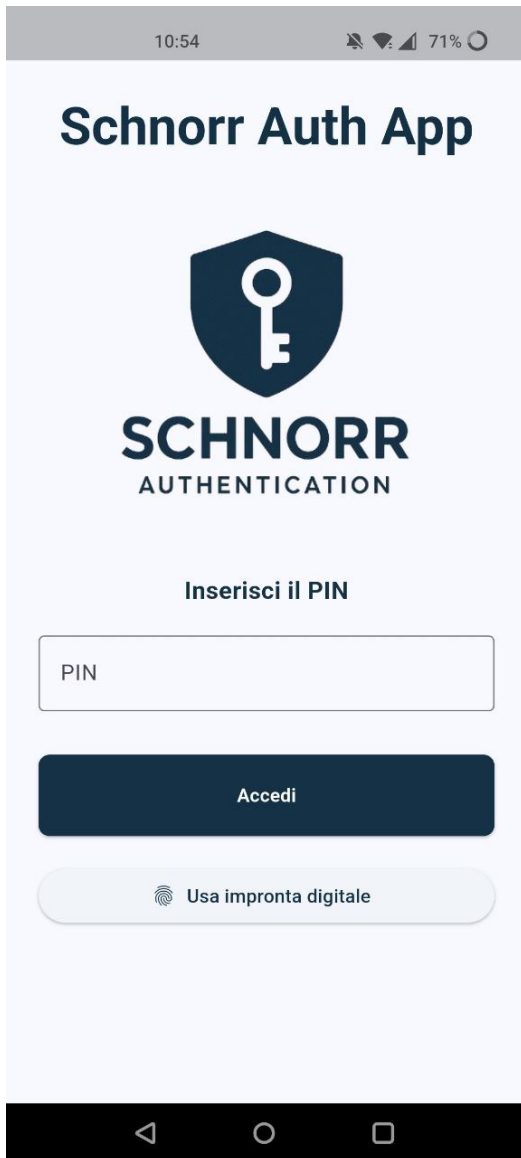


Figura 3.5: Schermata iniziale nella quale impostare il PIN.

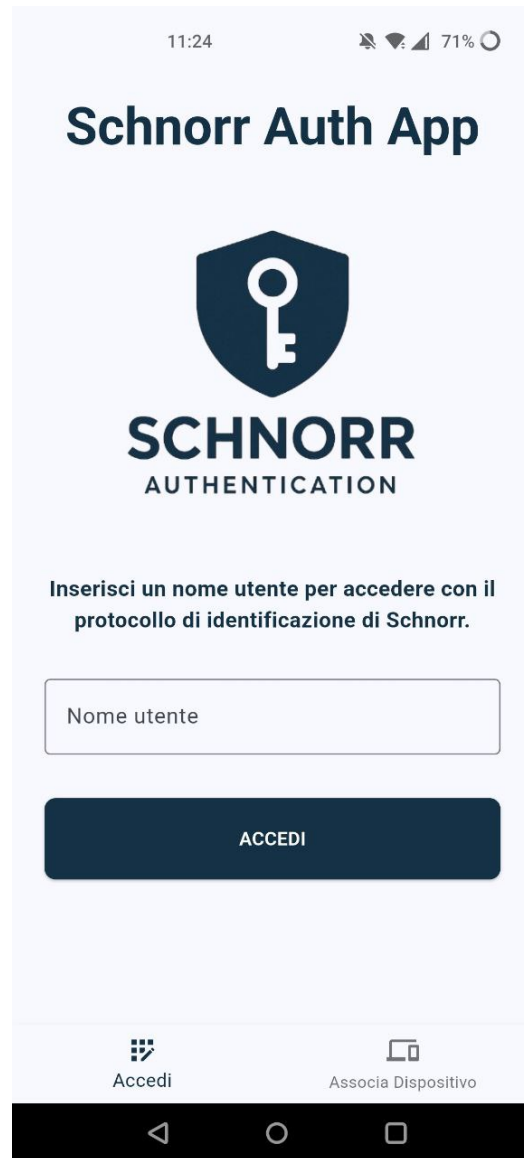


Figura 3.6: Fase di registrazione/autenticazione.

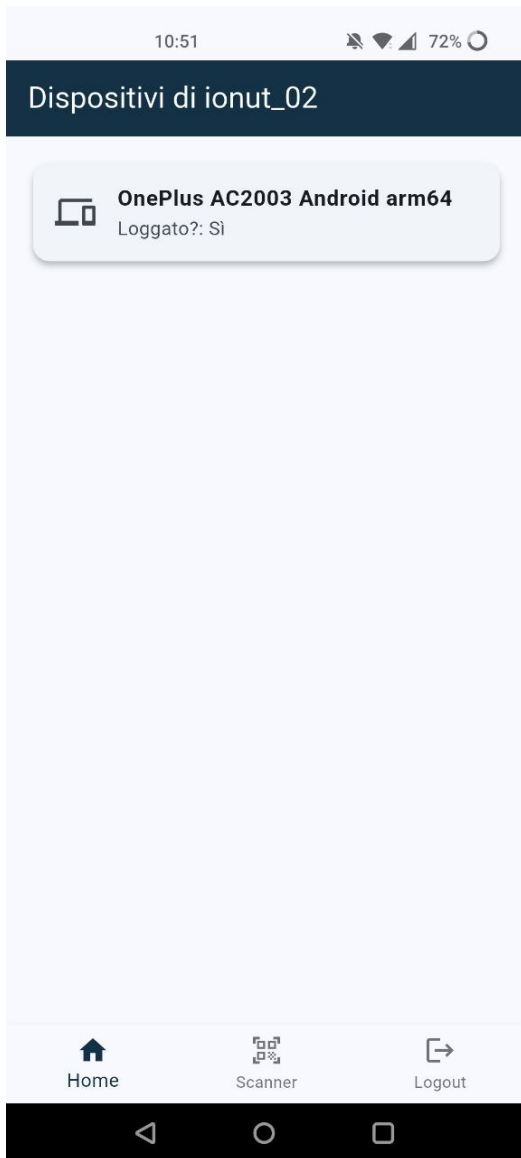


Figura 3.7: Pagina principale con dispositivi associati.



Figura 3.8: Pagina con QR code e parole.

Capitolo 4

Conclusioni e Sviluppi Futuri

Il presente capitolo ha la funzione di raccogliere le considerazioni conclusive relative al lavoro di ricerca e sviluppo svolto. L'elaborato si è mosso a partire da una fase di studio teorico sui protocolli di *Zero Knowledge Proof*, passando in seguito alla loro sottoclasse dei protocolli Sigma, per poi concentrarsi in maniera specifica sul protocollo di identificazione di Schnorr. Di quest'ultimo sono stati analizzati sia il meccanismo di autenticazione interattiva, sia l'adattamento che consente di derivarne uno schema di firma digitale. Tale percorso ha fornito le basi concettuali necessarie per la progettazione e realizzazione dell'applicazione *SchnorrAuthApp*, che integra questi principi in un contesto applicativo concreto.

Nonostante l'elevato livello di sicurezza garantito da un protocollo di autenticazione basato su meccanismi di *Zero Knowledge Proof* (ZKP), la sua adozione in contesti reali risulta ancora complessa. I protocolli di autenticazione standard, come PAP o CHAP, continuano infatti a essere ampiamente utilizzati, poiché offrono un compromesso ritenuto accettabile tra sicurezza, semplicità di implementazione e facilità d'uso. Di conseguenza, i protocolli ZKP, pur rappresentando una soluzione tecnologicamente più avanzata, faticano a trovare un impiego diffuso al di fuori di scenari sperimentali o ambiti in cui la sicurezza riveste un'importanza critica.

L'applicazione sviluppata rappresenta comunque un risultato significativo, poiché dimostra la fattibilità tecnica di un sistema di autenticazione fondato su principi crittografici avanzati. *SchnorrAuthApp* traduce in termini concreti e operativi un modello teorico complesso, mostrando come i protocolli ZKP possano essere implementati in modo efficace anche in ambienti mobili e multiplatforma, grazie al framework Flutter. La pubblicazione del codice sorgente su una piattaforma open source come GitLab contribuisce inoltre alla trasparenza e alla riproducibilità del lavoro, favorendo la collaborazione e la possibilità di ulteriori evoluzioni da parte della comunità.

Durante lo sviluppo sono emerse tuttavia alcune sfide tecniche e concettuali. Una delle principali difficoltà è legata alla gestione sicura delle chiavi crittografiche, in

particolare nei contesti mobili dove la persistenza dei dati e la protezione dell'ambiente di esecuzione non sono sempre garantite. Allo stesso modo, l'esigenza di bilanciare sicurezza e usabilità rappresenta un aspetto centrale: un sistema di autenticazione, per essere realmente adottato, deve infatti risultare semplice da usare e da integrare con le infrastrutture esistenti. Inoltre, la mancanza di standard consolidati per l'integrazione dei protocolli ZKP nei framework di autenticazione più diffusi (come OAuth 2.0 o OpenID Connect) costituisce un ulteriore ostacolo alla loro diffusione su larga scala.

4.1 Possibili Sviluppi Futuri

Gli sviluppi futuri del progetto potranno concentrarsi su diverse direzioni. Un primo passo potrà consistere nell'integrazione del protocollo di Schnorr con architetture di autenticazione standard, in modo da ampliare la compatibilità del sistema con applicazioni web e servizi distribuiti. Un'evoluzione naturale del progetto potrebbe consistere nell'integrazione del sistema di autenticazione basato sul protocollo di Schnorr all'interno di un server web, come componente o plugin di librerie e framework HTTP comunemente utilizzati nello sviluppo di applicazioni web. In questo modo, il protocollo potrebbe essere impiegato non solo per l'autenticazione degli utenti, ma anche per la protezione delle API, offrendo un'alternativa sicura ai tradizionali meccanismi basati su token o password. Una tale integrazione permetterebbe di estendere l'utilizzo del sistema a contesti reali e distribuiti, sfruttando l'interoperabilità offerta dai principali linguaggi di programmazione e framework moderni.

Un'ulteriore estensione potrebbe riguardare l'introduzione di una funzionalità per l'esportazione e l'importazione sicura della chiave privata, utile a creare copie di backup protette. In questo modo, un utente che dispone di un solo dispositivo avrebbe la possibilità di salvare la propria chiave segreta su un supporto esterno, come un hard drive, senza comprometterne la sicurezza. Tale meccanismo garantirebbe una maggiore resilienza ai guasti o alla perdita del dispositivo principale, mantenendo al contempo la riservatezza della chiave segreta.

Bibliografia

- [1] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.
- [2] C.-P. Schnorr, “Efficient signature generation by smart cards,” *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.
- [3] M. Kojo and T. Kivinen, “More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE).” RFC 3526, May 2003.
- [4] D. R. Stinson and M. B. Paterson, *Cryptography: Theory and Practice*. CRC Press, 4th ed., 2019.
- [5] J. Daemen and V. Rijmen, *The Design of Rijndael: AES - The Advanced Encryption Standard*. 01 2002.
- [6] J. A. Salowey, D. McGrew, and A. Choudhury, “AES Galois Counter Mode (GCM) Cipher Suites for TLS.” RFC 5288, Aug. 2008.
- [7] Q. H. Dang, “Secure hash standard (shs),” federal information processing standards (fips) publication 180-4, National Institute of Standards and Technology (NIST), Gaithersburg, MD, 2015.
- [8] J.-J. Quisquater, M. Quisquater, M. Quisquater, M. Quisquater, L. Guillou, M. A. Guillou, G. Guillou, A. Guillou, G. Guillou, and S. Guillou, “How to explain zero-knowledge protocols to your children,” in *Advances in Cryptology — CRYPTO’ 89 Proceedings* (G. Brassard, ed.), (New York, NY), pp. 628–631, Springer New York, 1990.
- [9] R. Cramer, *Modular Design of Secure yet Practical Cryptographic Protocols*. PhD thesis, Jan. 1997.
- [10] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*. 2020. Copia disponibile a questo link: <https://toc.cryptobook.us/>.

- [11] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology — CRYPTO’ 86* (A. M. Odlyzko, ed.), (Berlin, Heidelberg), pp. 186–194, Springer Berlin Heidelberg, 1987.
- [12] Python Software Foundation, “Python: A dynamic, open source programming language,” 2025. Documentazione disponibile a questo link: <https://www.python.org/>.
- [13] MongoDB Inc., “Mongodb: The developer data platform,” 2025. Documentazione disponibile a questo link: <https://www.mongodb.com/>.
- [14] W. Eddy, “Transmission Control Protocol (TCP).” RFC 9293, Aug. 2022. Documento disponibile a questo link: <https://www.rfc-editor.org/info/rfc9293>.
- [15] Google, “Flutter: Build apps for any screen,” 2025. Documentazione disponibile a questo link: <https://flutter.dev/>.
- [16] R. Right, *Beanie Documentation*, 2025. Documentazione disponibile a questo link: <https://docs.pydantic.dev/>.
- [17] S. Colvin and P. Team, *Pydantic Documentation*, 2025. Documentazione disponibile a questo link: <https://docs.pydantic.dev/>.
- [18] M. Palatinus, P. Rusnak, A. Voisine, and S. Bowe, “Bip 39: Mnemonic code for generating deterministic keys.” <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>, Sept. 2013. Status: Final. Layer: Applications. License: MIT.
- [19] Dart Project, “Dart programming language,” 2011. Documentazione disponibile a questo link: <https://www.dart.dev/>.
- [20] F-Droid Project, “F-droid: Free and open source android app repository.” <https://f-droid.org/>, 2010.

Appendice A

Codice sorgente

Questa appendice raccoglie il codice sorgente principale relativo alle componenti implementative del protocollo di autenticazione basato su Schnorr. Le classi riportate rappresentano i moduli centrali per la gestione delle chiavi crittografiche e per l'esecuzione delle fasi di prova e verifica all'interno del protocollo.

A.1 KeyManager

La classe `KeyManager` gestisce la memorizzazione sicura delle chiavi private degli utenti. Se viene fornita una password, la chiave viene cifrata con AES-GCM utilizzando una chiave derivata tramite Scrypt. In assenza di password, la chiave viene salvata in chiaro nel file system locale.

```

1  import os
2  import cryptography
3  from pathlib import Path
4  from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
5  from cryptography.hazmat.primitives.ciphers.aead import AESGCM
6
7  class KeyManager:
8      HOME_PATH = Path.home()
9      SCHNORR_DIR = HOME_PATH / ".schnorr"
10
11     @staticmethod
12     def _derive_key(password: str, salt: bytes) -> bytes:
13         """Deriva una chiave AES-256 (32 byte) usando PBKDF2-HMAC-SHA256"""
14         ...
15
16     @classmethod
17     def save_private_key(cls, username: str, key: int, password: str = None) -> None:
18         cls.SCHNORR_DIR.mkdir(parents=True, exist_ok=True)
19         privkey_path = cls.SCHNORR_DIR / f"{username}_privkey.pem"
20
21         if not password:
22             cls._save_private_key(username, key)
23             return
24
25         data = str(key).encode()
26         salt, nonce = os.urandom(16), os.urandom(12)
27         aesgcm = AESGCM(cls._derive_key(password, salt))
28         ciphertext = aesgcm.encrypt(nonce, data, None)
29         with open(privkey_path, "wb") as f:
30             f.write(salt + nonce + ciphertext)
31
32     @classmethod
33     def load_private_key(cls, username: str, password: str = None) -> int:
34         privkey_path = cls.SCHNORR_DIR / f"{username}_privkey.pem"
35
36         if not password:
37             return cls._load_private_key(username)
38
39         with open(privkey_path, "rb") as f:
40             blob = f.read()
41             salt, nonce, ciphertext = blob[:16], blob[16:28], blob[28:]
42             aesgcm = AESGCM(cls._derive_key(password, salt))
43             plaintext = aesgcm.decrypt(nonce, ciphertext, None)
44             return int(plaintext.decode())

```

Codice 4: Implementazione della classe KeyManager.

A.2 SchnorrProver e SchnorrVerifier

Le classi SchnorrProver e SchnorrVerifier implementano rispettivamente il ruolo del Prover e del Verifier nel protocollo di identificazione di Schnorr. La prima gestisce

la generazione delle chiavi e la produzione della risposta, mentre la seconda verifica la validità della prova o della firma associata.

```

1  class Schnorr:
2      def __init__(self, group_id: GroupType):
3          self.crypto_group = Rfc3526(group_id)
4          self.p, self.g, self.q = self.crypto_group.get_params()
5          self.public_key, self.public_key_temp = None, None
6
7      def compute_challenge(self, message: int) -> int:
8          data = str(self.public_key_temp) + str(self.public_key) + message
9          return int(hashlib.sha256(data.encode()).hexdigest(), 16) % self.q
10
11  class SchnorrProver(Schnorr):
12      def __init__(self, group_id: GroupType):
13          super().__init__(group_id, filename)
14          self.alpha, self.alpha_temp = None, None
15
16      def public_key(self) -> int:
17          if self.public_key is None:
18              self.public_key = pow(self.g, self.alpha, self.p)
19          return self.public_key
20
21      def public_key_temp(self) -> int:
22          self.alpha_temp = random.randint(1, self.q - 1)
23          self.public_key_temp = pow(self.g, self.alpha_temp, self.p)
24          return self.public_key_temp
25
26      def gen_keys(self):
27          self.alpha = random.randint(1, self.q - 1)
28          self.public_key = pow(self.g, self.alpha, self.p)
29
30      def response(self, challenge: int) -> int:
31          return (self.alpha_temp + self.alpha * challenge) % self.q
32
33      def sign_message(self, message: str) -> dict[str, int]:
34          challenge = self.compute_challenge(message)
35          sign = {"public_key_temp": self.public_key_temp, "response": self.response(challenge)}
36          return sign
37
38  class SchnorrVerifier(Schnorr):
39      def __init__(self, group_id: GroupType):
40          super().__init__(group_id, filename)
41          self.challenge = random.randint(0, self.q - 1)
42
43      def check(self, response: int) -> bool:
44          left = pow(self.g, response, self.p)
45          right = (self.public_key_temp * pow(self.public_key, self.challenge, self.p)) % self.p
46          return left == right
47
48      def verify_sign(self, sign: dict, message: str) -> bool:
49          self.public_key_temp = sign["public_key_temp"]
50          self.challenge = self.compute_challenge(message)
51          return self.check(sign["response"])

```

Codice 5: Classi SchnorrProver e SchnorrVerifier.